Cours #1 and Lab #1 - Introduction to Neural Networks

Prof. Abdelouahab Moussaoui

Academic Year: 2005-2006

For: 4th Year Engineering Students
Option: Artificial Intelligence

Abstract

This practical work introduces the historical foundations of neural networks, from the McCulloch-Pitts neuron (1943) to ADALINE (1960). These models, developed well before the era of deep learning, form the conceptual basis of modern artificial neural networks. Throughout this lab, you will implement these models from scratch, understand their learning rules mathematically, and explore their capabilities and limitations through theoretical exercises and practical Python implementations.

Important Note: This is not about deep learning. We are studying the classical neural networks that laid the groundwork for today's advanced architectures. Understanding these fundamentals is essential before moving to more complex models.

Contents

1 General Introduction **4**
- 1.1 What is an Artificial Neural Network? . . . 4
- 1.2 Timeline of Pioneers . . . 4
- 1.3 What We Will Study . . . 4

2 McCulloch-Pitts Neuron (1943) **5**
- 2.1 Historical Introduction . . . 5
  - 2.1.1 The Model . . . 5
  - 2.1.2 Important Properties . . . 5
  - 2.1.3 Fundamental Limitation . . . 5
- 2.2 Theoretical Exercises . . . 5
- 2.3 Implementing Logic Gates with McCulloch-Pitts Neuron . . . 7

2.3.1 AND Gate . . . 8
2.3.2 OR Gate . . . 8
2.3.3 NOT Gate (Unary) . . . 9
2.3.4 NAND Gate . . . 9
2.3.5 NOR Gate . . . 10
2.3.6 XOR Gate (Impossible with Single Neuron) . . . 10
2.3.7 Demonstration of XOR Impossibility . . . 12
2.3.8 Complete Test Function . . . 12
2.3.9 Expected Output . . . 13
2.4 Visualizing Decision Boundaries . . . 14
2.5 Summary . . . 15
2.6 Additional Exercises . . . 15

3 Hebb's Rule (1949) 16
3.1 Historical Introduction . . . 16
3.1.1 Hebb's Principle . . . 16
3.1.2 Mathematical Formulation . . . 16
3.1.3 Important Characteristics . . . 16
3.1.4 Applications . . . 16
3.2 Theoretical Exercises . . . 17
3.3 Implementation of Hebbian Learning in Python . . . 18
3.3.1 Basic Hebbian Neuron Implementation . . . 18
3.3.2 Learning Simple Patterns . . . 19
3.3.3 Learning Logic Gates with Hebbian Rule . . . 20
3.3.4 Observations on Hebbian Learning of Logic Gates . . . 21
3.3.5 Effect of Learning Rate on Hebbian Learning . . . 21
3.3.6 Problem of Weight Divergence . . . 22
3.3.7 Oja's Rule: A Normalized Solution . . . 23
3.3.8 Hebbian Learning as Principal Component Analysis . . . 25
3.3.9 Simple Associative Memory with Hebbian Learning . . . 26
3.3.10 Visualizing Weight Evolution During Learning . . . 28
3.4 Summary of Hebbian Learning . . . 30
3.5 Additional Exercises . . . 30

4 Perceptron (Rosenblatt, 1957) 31
4.1 Historical Introduction . . . 31
4.1.1 Context . . . 31
4.1.2 The Perceptron Model . . . 31
4.1.3 Mathematical Formulation . . . 31
4.1.4 Properties . . . 32
4.1.5 Limitations . . . 32
4.2 Theoretical Exercises . . . 32
4.3 Implementation of Perceptron in Python . . . 34
4.3.1 Basic Perceptron Implementation . . . 34
4.3.2 Learning Logic Gates with Perceptron . . . 36
4.3.3 Observations on XOR . . . 38
4.3.4 Effect of Learning Rate . . . 39
4.3.5 Decision Boundary Visualization . . . 41
4.3.6 Effect of Data Ordering . . . 43

4.3.7 Voting Perceptron . . . 44
4.3.8 Perceptron on Real Dataset: Iris . . . 46
4.3.9 Perceptron with Non-Separable Data . . . 48
4.3.10 Comparison with Scikit-Learn Perceptron . . . 49
4.4 Summary of Perceptron . . . 50
4.5 Additional Exercises . . . 51

5 ADALINE (Widrow & Hoff, 1960) 53
5.1 Historical Introduction . . . 53
5.1.1 Context . . . 53
5.1.2 Major Innovations . . . 53
5.1.3 Key Difference from Perceptron . . . 53
5.1.4 Mathematical Formulation . . . 54
5.2 Theoretical Exercises . . . 54
5.3 Implementation of ADALINE in Python . . . 56
5.3.1 Basic ADALINE Implementation . . . 56
5.3.2 Learning Logic Gates with ADALINE . . . 59
5.3.3 Comparing ADALINE and Perceptron on XOR . . . 61
5.3.4 Effect of Learning Rate on ADALINE . . . 62
5.3.5 Importance of Standardization . . . 64
5.3.6 Batch vs Stochastic vs Mini-Batch Gradient Descent . . . 65
5.3.7 ADALINE for Regression . . . 67
5.3.8 ADALINE on Iris Dataset . . . 69
5.3.9 Learning with Momentum . . . 72
5.3.10 Learning Rate Decay . . . 74
5.4 Summary of ADALINE . . . 77
5.5 Additional Exercises . . . 77

1 General Introduction

1.1 What is an Artificial Neural Network?

An artificial neural network is a computational model inspired by the structure and function of the biological brain. It consists of interconnected elementary units called "neurons" that transmit signals to each other. The first models date back to the 1940s-1950s, long before the era of "deep learning".

1.2 Timeline of Pioneers

Year	Researchers	Contribution
1943	McCulloch & Pitts	First mathematical model of a neuron
1949	Donald Hebb	Hebbian learning rule
1957	Frank Rosenblatt	The Perceptron
1960	Widrow & Hoff	ADALINE
1969	Minsky & Papert	Critique of the Perceptron
1982	John Hopfield	Hopfield networks
1986	Rumelhart, Hinton, Williams	Backpropagation

Table 1: Key milestones in the history of neural networks

1.3 What We Will Study

We will focus on the fundamental models:

1. **McCulloch-Pitts neuron:** The first model, without learning
2. **Hebb's rule:** The first unsupervised learning mechanism
3. **Perceptron:** The first trainable linear classifier
4. **ADALINE:** The first model using gradient descent

Important: Before starting, make sure you have installed the necessary Python libraries: `numpy`, `matplotlib`, `scikit-learn`. You can install them with: `pip install numpy matplotlib scikit-learn seaborn`

2 McCulloch-Pitts Neuron (1943)

2.1 Historical Introduction

In 1943, neurophysiologist Warren McCulloch and mathematician Walter Pitts published a revolutionary paper: "A Logical Calculus of the Ideas Immanent in Nervous Activity". Their goal was to understand whether the brain could be considered a logical machine.

2.1.1 The Model

The McCulloch-Pitts neuron is a simple but fundamental binary model:

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{n} w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where:

- $x_i \in \{0, 1\}$ are binary inputs
- w_i are fixed synaptic weights (set by the designer)
- θ is the activation threshold
- $y \in \{0, 1\}$ is the binary output

2.1.2 Important Properties

- **No learning:** weights are fixed
- **Binary inputs and outputs**
- **Positive (excitatory) or negative (inhibitory) weights**
- A single neuron can perform simple logical operations

2.1.3 Fundamental Limitation

A single McCulloch-Pitts neuron can only solve **linearly separable** problems (like AND, OR, NOT), but cannot solve non-linear problems like XOR.

2.2 Theoretical Exercises

Exercise 1.1 First manual calculation: Calculate the output of a McCulloch-Pitts neuron with:

- Weights = [1, 1, 1]
- Threshold = 2.5
- Inputs = [1, 0, 1]

Exercise 1.2 AND gate: Find the weights and threshold for the logical AND function.

Exercise 1.3 OR gate: Find the weights and threshold for the OR function.

Exercise 1.4 NAND gate: Find the weights and threshold for the NAND function.

Exercise 1.5 NOR gate: Find the weights and threshold for the NOR function.

Exercise 1.6 XOR impossibility: Show that XOR is impossible with a single neuron.

2.3 Implementing Logic Gates with McCulloch-Pitts Neuron

In this section, we will implement various logic gates using the McCulloch-Pitts neuron model. For each gate, we will:

- Present the truth table
- Determine the appropriate weights and threshold
- Implement the gate in Python
- Test all possible input combinations

First, we need to implement the base McCulloch-Pitts neuron class:

```python
import numpy as np
import matplotlib.pyplot as plt

class McCullochPittsNeuron:
    """McCulloch-Pitts neuron with fixed weights and threshold"""

    def __init__(self, weights, threshold):
        """
        Initialize the neuron with fixed weights and threshold

        Parameters:
        weights (list): Synaptic weights
        threshold (float): Activation threshold
        """
        self.weights = np.array(weights)
        self.threshold = threshold

    def activate(self, inputs):
        """
        Activate the neuron for given inputs

        Parameters:
        inputs (list): Input values (binary)

        Returns:
        int: 1 if weighted sum >= threshold, else 0
        """
        weighted_sum = np.dot(inputs, self.weights)
        return 1 if weighted_sum >= self.threshold else 0

    def predict(self, X):
        """
        Predict outputs for multiple input samples

        Parameters:
        X (array-like): Matrix of input samples

        Returns:
```

```
        array: Predictions for all samples
        """
        return np.array([self.activate(x) for x in X])
```

Listing 1: Basic McCulloch-Pitts neuron implementation

2.3.1 AND Gate

The AND gate returns 1 only when both inputs are 1.

Input A	Input B	Output (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

Table 2: Truth table for AND gate

Configuration: Weights = [1, 1], Threshold = 2

```
def and_gate():
    """Return a McCulloch-Pitts neuron configured as AND gate"""
    return McCullochPittsNeuron([1, 1], 2)

# Test AND gate
and_neuron = and_gate()
print("AND Gate Truth Table:")
print("A B | Output")
print("----+-------")
for a in [0, 1]:
    for b in [0, 1]:
        output = and_neuron.activate([a, b])
        print(f"{a} {b} |   {output}")
```

2.3.2 OR Gate

The OR gate returns 1 if at least one input is 1.

Input A	Input B	Output (A OR B)
0	0	0
0	1	1
1	0	1
1	1	1

Table 3: Truth table for OR gate

Configuration: Weights = [1, 1], Threshold = 1

```
def or_gate():
    """Return a McCulloch-Pitts neuron configured as OR gate"""
    return McCullochPittsNeuron([1, 1], 1)
```

```

# Test OR gate
or_neuron = or_gate()
print("\nOR Gate Truth Table:")
print("A B | Output")
print("----+-------")
for a in [0, 1]:
    for b in [0, 1]:
        output = or_neuron.activate([a, b])
        print(f"{a} {b} |   {output}")
```

2.3.3 NOT Gate (Unary)

The NOT gate (inverter) returns the opposite of the input.

Input A	Output (NOT A)
0	1
1	0

Table 4: Truth table for NOT gate

Configuration: Weights = [-1], Threshold = 0

```
def not_gate():
    """Return a McCulloch-Pitts neuron configured as NOT gate"""
    return McCullochPittsNeuron([-1], 0)

# Test NOT gate
not_neuron = not_gate()
print("\nNOT Gate Truth Table:")
print("A | Output")
print("--+-------")
for a in [0, 1]:
    output = not_neuron.activate([a])
    print(f"{a} |   {output}")
```

2.3.4 NAND Gate

The NAND gate (NOT AND) returns 0 only when both inputs are 1.

Input A	Input B	Output (A NAND B)
0	0	1
0	1	1
1	0	1
1	1	0

Table 5: Truth table for NAND gate

Configuration: Weights = [-1, -1], Threshold = -1.5

```python
def nand_gate():
    """Return a McCulloch-Pitts neuron configured as NAND gate"""
    return McCullochPittsNeuron([-1, -1], -1.5)

# Test NAND gate
nand_neuron = nand_gate()
print("\nNAND Gate Truth Table:")
print("A B | Output")
print("----+-------")
for a in [0, 1]:
    for b in [0, 1]:
        output = nand_neuron.activate([a, b])
        print(f"{a} {b} |   {output}")
```

2.3.5 NOR Gate

The NOR gate (NOT OR) returns 1 only when both inputs are 0.

Input A	Input B	Output (A NOR B)
0	0	1
0	1	0
1	0	0
1	1	0

Table 6: Truth table for NOR gate

Configuration: Weights = [-1, -1], Threshold = -0.5

```python
def nor_gate():
    """Return a McCulloch-Pitts neuron configured as NOR gate"""
    return McCullochPittsNeuron([-1, -1], -0.5)

# Test NOR gate
nor_neuron = nor_gate()
print("\nNOR Gate Truth Table:")
print("A B | Output")
print("----+-------")
for a in [0, 1]:
    for b in [0, 1]:
        output = nor_neuron.activate([a, b])
        print(f"{a} {b} |   {output}")
```

2.3.6 XOR Gate (Impossible with Single Neuron)

The XOR gate returns 1 when inputs are different.

As we have seen, it is impossible to implement XOR with a single McCulloch-Pitts neuron because the problem is not linearly separable. However, we can implement it using a network of neurons.

Input A	Input B	Output (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

Table 7: Truth table for XOR gate

```python
class XORNetwork:
    """Network of 3 McCulloch-Pitts neurons for XOR"""

    def __init__(self):
        # Neuron 1: A AND NOT B
        self.n1 = McCullochPittsNeuron([1, -1], 1)
        # Neuron 2: NOT A AND B
        self.n2 = McCullochPittsNeuron([-1, 1], 1)
        # Neuron 3: OR of both
        self.n3 = McCullochPittsNeuron([1, 1], 1)

    def predict(self, inputs):
        """
        Predict XOR output for given inputs

        Parameters:
        inputs: list [a, b] with a,b in {0,1}

        Returns:
        int: XOR result
        """
        out1 = self.n1.activate(inputs)
        out2 = self.n2.activate(inputs)
        return self.n3.activate([out1, out2])

    def evaluate(self):
        """Test all input combinations"""
        test_inputs = [[0,0], [0,1], [1,0], [1,1]]
        expected = [0, 1, 1, 0]

        print("XOR Network Evaluation:")
        for inputs, exp in zip(test_inputs, expected):
            pred = self.predict(inputs)
            status = "\\cmark" if pred == exp else "\\xmark"
            print(f"  {inputs[0]} XOR {inputs[1]} = {pred} {
                status}")

# Test the XOR network
xor_net = XORNetwork()
xor_net.evaluate()
```

2.3.7 Demonstration of XOR Impossibility

```python
# Demonstration of XOR impossibility with a single neuron
print("\nXOR Gate - Why it's impossible with a single neuron:")
print("A B | Expected | Condition")
print("----+----------+----------")
points = [(0,0,0), (0,1,1), (1,0,1), (1,1,0)]
for a, b, expected in points:
    if expected == 1:
        condition = f"w1*{a} + w2*{b} >= theta"
    else:
        condition = f"w1*{a} + w2*{b} < theta"
    print(f"{a} {b} |    {expected}     | {condition}")
```

2.3.8 Complete Test Function

```python
def test_all_gates():
    """Test all implemented logic gates"""

    gates = {
        "AND": and_gate(),
        "OR": or_gate(),
        "NOT": not_gate(),
        "NAND": nand_gate(),
        "NOR": nor_gate()
    }

    # Test inputs for 2-input gates
    test_inputs_2 = [[0,0], [0,1], [1,0], [1,1]]

    # Test inputs for NOT gate (1-input)
    test_inputs_1 = [[0], [1]]

    # Expected outputs
    expected = {
        "AND": [0, 0, 0, 1],
        "OR": [0, 1, 1, 1],
        "NOT": [1, 0],
        "NAND": [1, 1, 1, 0],
        "NOR": [1, 0, 0, 0]
    }

    print("=" * 60)
    print("TESTING ALL LOGIC GATES")
    print("=" * 60)

    for gate_name, neuron in gates.items():
        print(f"\n{gate_name} Gate:")
        print("-" * 40)

```

```
        if gate_name == "NOT":
            inputs = test_inputs_1
        else:
            inputs = test_inputs_2

        for i, inp in enumerate(inputs):
            output = neuron.activate(inp)
            exp = expected[gate_name][i]
            status = "OK" if output == exp else "FAIL"
            if gate_name == "NOT":
                print(f"  NOT {inp[0]} = {output} [{status}]")
            else:
                print(f"  {inp[0]} {gate_name} {inp[1]} = {output
                    } [{status}]")

# Run the test
test_all_gates()
```

2.3.9 Expected Output

When you run the complete test, you should see:

```
============================================================
TESTING ALL LOGIC GATES
============================================================

AND Gate:
----------------------------------------
  0 AND 0 = 0 [OK]
  0 AND 1 = 0 [OK]
  1 AND 0 = 0 [OK]
  1 AND 1 = 1 [OK]

OR Gate:
----------------------------------------
  0 OR 0 = 0 [OK]
  0 OR 1 = 1 [OK]
  1 OR 0 = 1 [OK]
  1 OR 1 = 1 [OK]

NOT Gate:
----------------------------------------
  NOT 0 = 1 [OK]
  NOT 1 = 0 [OK]

NAND Gate:
----------------------------------------
  0 NAND 0 = 1 [OK]
  0 NAND 1 = 1 [OK]
```

```
  1 NAND 0 = 1 [OK]
  1 NAND 1 = 0 [OK]

NOR Gate:
----------------------------------------
  0 NOR 0 = 1 [OK]
  0 NOR 1 = 0 [OK]
  1 NOR 0 = 0 [OK]
  1 NOR 1 = 0 [OK]
```

2.4 Visualizing Decision Boundaries

For 2-input gates, we can visualize the decision boundary:

```python
def plot_gate_boundary(neuron, gate_name):
    """Plot the decision boundary for a 2-input logic gate"""

    # Create a grid of points
    x = np.linspace(-0.2, 1.2, 100)
    y = np.linspace(-0.2, 1.2, 100)
    X, Y = np.meshgrid(x, y)

    # Compute predictions
    Z = np.zeros_like(X)
    for i in range(len(x)):
        for j in range(len(y)):
            Z[i,j] = neuron.activate([X[i,j], Y[i,j]])

    # Plot
    plt.figure(figsize=(8, 6))
    plt.contourf(X, Y, Z, alpha=0.3, levels=[-0.5, 0.5, 1.5],
       colors=['blue', 'red'])
    plt.contour(X, Y, Z, levels=[0.5], colors='black', linewidths
       =2)

    # Plot the four corners
    corners = [[0,0], [0,1], [1,0], [1,1]]
    for x1, x2 in corners:
        color = 'red' if neuron.activate([x1, x2]) == 1 else '
           blue'
        plt.scatter(x1, x2, c=color, s=200, edgecolors='black',
           linewidth=2)
        plt.text(x1-0.05, x2+0.05, f'({x1},{x2})', fontsize=10)

    plt.xlabel('Input A')
    plt.ylabel('Input B')
    plt.title(f'{gate_name} Gate - Decision Boundary')
    plt.grid(True, alpha=0.3)
    plt.xlim(-0.2, 1.2)
    plt.ylim(-0.2, 1.2)
    plt.show()
```

```
34
35 # Plot all gates
36 gates_to_plot = {
37     "AND": and_gate(),
38     "OR": or_gate(),
39     "NAND": nand_gate(),
40     "NOR": nor_gate()
41 }
42
43 for name, neuron in gates_to_plot.items():
44     plot_gate_boundary(neuron, name)
```

2.5 Summary

We have successfully implemented all basic logic gates using the McCulloch-Pitts neuron:

- **AND, OR, NOT, NAND, NOR** can be implemented with a single neuron
- **XOR** requires a network of multiple neurons
- The decision boundaries show that each gate creates a linear separation in the input space
- This demonstrates the power and limitations of single neurons

2.6 Additional Exercises

Exercise 1.8 **3-input AND gate:** Implement a 3-input AND gate using a McCulloch-Pitts neuron.

Exercise 1.9 **Majority gate:** Implement a 3-input gate that outputs 1 if at least 2 inputs are 1.

Exercise 1.10 **Specific pattern detector:** Implement a neuron that detects pattern [1,0,1,0].

Exercise 1.11 **Combined gates:** Use your AND and NOT gates to build a NAND gate (verifying that NAND = NOT(AND)).

3 Hebb's Rule (1949)

3.1 Historical Introduction

In 1949, Canadian psychologist Donald Hebb published "The Organization of Behavior", a work proposing a neuropsychological theory of learning. His major contribution is a simple but powerful rule describing how connections between neurons strengthen.

3.1.1 Hebb's Principle

Often summarized as "**cells that fire together, wire together**", Hebb's original statement was:

> "When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased."

3.1.2 Mathematical Formulation

The change in synaptic weight is proportional to the product of pre-synaptic activity (input) and post-synaptic activity (output):

$$\Delta w_i = \eta \times x_i \times y \tag{2}$$

where:

- Δw_i is the change in weight i
- η is the learning rate (positive constant)
- x_i is the input (pre-synaptic activity)
- y is the neuron's output (post-synaptic activity)

3.1.3 Important Characteristics

- **Unsupervised learning:** no external error signal
- **Local:** modification depends only on connected neurons
- **Hebbian:** reinforcement of correlations
- **Instability:** weights can grow indefinitely

3.1.4 Applications

Associative memory, correlation detection, pattern learning.

3.2 Theoretical Exercises

Exercise 2.1 **Understanding the formula:** Explain in your own words why Hebb's rule strengthens connections between neurons that fire together.

Exercise 2.2 **First calculation:** A neuron has initial weight w = 0.5. Input x = 1 is presented, and the neuron gives output y = 0.8. With eta = 0.1, calculate the new weight.

Exercise 2.3 **Extreme cases:** What happens if:

- x = 0 (zero input)?
- y = 0 (inactive neuron)?

Exercise 2.4 **Multiple steps evolution:** Initial weight = 0.2, eta = 0.05. The same pattern (x=1, y=0.9) is presented 5 times. Calculate the weight evolution.

Exercise 2.5 **Divergence problem:** With eta = 0.1 and x = y = 1 at each step, how many steps are needed for the weight to exceed 10 starting from 0?

Exercise 2.6 **Pattern learning:** A neuron receives 3 inputs. Initially all weights are 0.1. The pattern [1,0,1] is presented 10 times, and each time the output is 1. Calculate the final weights (eta = 0.01).

Exercise 2.7 **Implicit competition:** Explain why, with Hebb's rule, inputs that are often active together see their weights increase faster.

Exercise 2.8 **Forgetting:** Hebb's rule has no forgetting mechanism. What problem does this pose?

Exercise 2.9 **Normalization:** Propose a modification to Hebb's rule to prevent weight divergence.

Exercise 2.10 **Anti-Hebb:** What would happen with negative eta? What could this be used for?

3.3 Implementation of Hebbian Learning in Python

In this section, we will implement a Hebbian neuron and explore its learning capabilities. Unlike McCulloch-Pitts, Hebbian neurons can learn from data without supervision.

3.3.1 Basic Hebbian Neuron Implementation

```python
import numpy as np
import matplotlib.pyplot as plt

class HebbianNeuron:
    """Neuron with Hebbian learning rule"""

    def __init__(self, n_inputs, learning_rate=0.01):
        """
        Initialize Hebbian neuron

        Parameters:
        n_inputs: number of inputs
        learning_rate: eta (learning rate)
        """
        self.lr = learning_rate
        # Small random initialization
        self.weights = np.random.randn(n_inputs) * 0.1
        self.bias = np.random.randn() * 0.1
        self.weight_history = []  # For visualization

    def activation(self, x):
        """Sigmoid activation function"""
        z = np.dot(x, self.weights) + self.bias
        return 1 / (1 + np.exp(-z))

    def predict(self, x):
        """Binary prediction (for after learning)"""
        return 1 if self.activation(x) > 0.5 else 0

    def hebbian_update(self, x, y):
        """
        Hebb's rule: delta w = eta * x * y

        Note: y is the neuron's output, not the target!
        """
        self.weights += self.lr * x * y
        self.bias += self.lr * y  # Bias sees constant input = 1

    def train(self, X, epochs=10, verbose=True):
        """
        Unsupervised Hebbian learning

        Parameters:
        X: input data
```

```
        epochs: number of passes through the data
        verbose: print progress
        """
        self.weight_history = []

        for epoch in range(epochs):
            for x in X:
                y = self.activation(x)
                self.hebbian_update(x, y)

            # Save for visualization
            self.weight_history.append(self.weights.copy())

            if verbose and (epoch + 1) % 5 == 0:
                print(f"Epoch {epoch+1}/{epochs}, Weights: {self.
                    weights}")

        return self
```

Listing 2: Hebbian neuron implementation

3.3.2 Learning Simple Patterns

Let's see how a Hebbian neuron learns to detect patterns in data:

```
# Create a simple dataset with structure
np.random.seed(42)

# Create two clusters of points
n_samples = 100
cluster1 = np.random.randn(n_samples, 2) * 0.5 + np.array([1, 1])
    # Around (1,1)
cluster2 = np.random.randn(n_samples, 2) * 0.5 + np.array([-1,
    -1])  # Around (-1,-1)
X = np.vstack([cluster1, cluster2])

# Train Hebbian neuron
hebb = HebbianNeuron(n_inputs=2, learning_rate=0.01)
hebb.train(X, epochs=30, verbose=True)

# Visualize the learning process
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
weight_history = np.array(hebb.weight_history)
plt.plot(weight_history[:, 0], label='Weight 1', linewidth=2)
plt.plot(weight_history[:, 1], label='Weight 2', linewidth=2)
plt.xlabel('Epoch')
plt.ylabel('Weight value')
plt.title('Hebbian Weight Evolution')
plt.legend()
plt.grid(True, alpha=0.3)
```

```
plt.subplot(1, 2, 2)
plt.scatter(X[:n_samples, 0], X[:n_samples, 1], alpha=0.5, label='Cluster 1')
plt.scatter(X[n_samples:, 0], X[n_samples:, 1], alpha=0.5, label='Cluster 2')

# Plot final weight direction
origin = np.array([0, 0])
plt.quiver(*origin, hebb.weights[0], hebb.weights[1],
           angles='xy', scale_units='xy', scale=3,
           color='red', label='Learned direction', width=0.02)

plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Data and Learned Direction')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.tight_layout()
plt.show()

print(f"\nFinal weights: {hebb.weights}")
print(f"The neuron has learned to point in the direction of maximum correlation")
```

3.3.3 Learning Logic Gates with Hebbian Rule

Can a Hebbian neuron learn logic gates? Let's experiment:

```
# Data for logic gates
X_logic = np.array([[0,0], [0,1], [1,0], [1,1]])
# For Hebbian learning, we don't provide targets!
# The neuron will learn based on correlations in the data

def train_hebbian_on_gate(X, gate_name, learning_rate=0.1, epochs=100):
    """Train a Hebbian neuron on logic gate inputs and analyze results"""

    hebb = HebbianNeuron(n_inputs=2, learning_rate=learning_rate)

    print(f"\nTraining Hebbian neuron on {gate_name} gate inputs")
    print("-" * 50)
    print("Initial weights:", hebb.weights)

    # Train on the inputs (no targets!)
    hebb.train(X, epochs=epochs, verbose=False)

    print("Final weights:", hebb.weights)
```

```
    # Test the neuron's responses to each input
    print("\nNeuron responses:")
    print("A B | Activation | Prediction")
    print("----+------------+-----------")
    for inputs in X:
        act = hebb.activation(inputs)
        pred = hebb.predict(inputs)
        print(f"{inputs[0]} {inputs[1]} |    {act:.4f}    |      {
            pred}")

    return hebb

# Train on AND gate inputs
hebb_and = train_hebbian_on_gate(X_logic, "AND")

# Train on OR gate inputs
hebb_or = train_hebbian_on_gate(X_logic, "OR")

# Train on XOR gate inputs
hebb_xor = train_hebbian_on_gate(X_logic, "XOR")
```

3.3.4 Observations on Hebbian Learning of Logic Gates

What do we observe? The Hebbian neuron does not learn the logic gates in the same way as a supervised algorithm:

- It learns the **statistical correlations** in the data
- For AND gate inputs, pattern (1,1) appears once, so it gets slightly higher weights
- For OR gate inputs, patterns with 1s appear more frequently, so weights increase more
- The neuron becomes more sensitive to inputs with many 1s
- It does **not** learn to separate classes - it learns to detect frequent patterns

3.3.5 Effect of Learning Rate on Hebbian Learning

```
def compare_hebbian_learning_rates(X, learning_rates=[0.001,
    0.01, 0.1, 1.0], epochs=50):
    """
    Compare Hebbian learning with different learning rates
    """
    plt.figure(figsize=(15, 10))

    results = {}

    for i, lr in enumerate(learning_rates):
        # Train neuron
```

```
        hebb = HebbianNeuron(n_inputs=X.shape[1], learning_rate=lr)
        hebb.train(X, epochs=epochs, verbose=False)

        # Plot weight evolution
        plt.subplot(2, 2, i+1)
        weight_history = np.array(hebb.weight_history)
        plt.plot(weight_history[:, 0], label='Weight 1', linewidth=2)
        plt.plot(weight_history[:, 1], label='Weight 2', linewidth=2)
        plt.xlabel('Epoch')
        plt.ylabel('Weight value')
        plt.title(f'Learning Rate = {lr}')
        plt.legend()
        plt.grid(True, alpha=0.3)

        # Store results
        results[lr] = {
            'final_weights': hebb.weights.copy(),
            'weight_norm': np.linalg.norm(hebb.weights)
        }

    plt.tight_layout()
    plt.show()

    # Print comparison
    print("\nLearning Rate Comparison:")
    print("-" * 50)
    for lr, res in results.items():
        print(f"LR = {lr}: Final weights = {res['final_weights']}, "
              f"Norm = {res['weight_norm']:.4f}")

    return results

# Generate correlated data
np.random.seed(42)
X_corr = np.random.multivariate_normal(
    mean=[0, 0],
    cov=[[1, 0.8], [0.8, 1]],
    size=200
)

results = compare_hebbian_learning_rates(X_corr)
```

3.3.6 Problem of Weight Divergence

One major issue with pure Hebbian learning is that weights can grow indefinitely:

```
# Demonstrate weight divergence
```

```
hebb_divergent = HebbianNeuron(n_inputs=2, learning_rate=0.1)

# Create data where all inputs are [1,1]
X_ones = np.ones((100, 2))

print("Training with constant inputs [1,1] - weights will diverge!")
hebb_divergent.train(X_ones, epochs=50, verbose=False)

plt.figure(figsize=(10, 5))
weight_history = np.array(hebb_divergent.weight_history)
plt.plot(weight_history[:, 0], label='Weight 1', linewidth=2)
plt.plot(weight_history[:, 1], label='Weight 2', linewidth=2)
plt.xlabel('Epoch')
plt.ylabel('Weight value')
plt.title('Weight Divergence in Pure Hebbian Learning')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(f"Final weights after 50 epochs: {hebb_divergent.weights}")
print("Weights keep growing indefinitely - this is unstable!")
```

3.3.7 Oja's Rule: A Normalized Solution

Oja's rule modifies Hebbian learning to prevent divergence by adding a normalization term:

$$\Delta w_i = \eta(x_i y - y^2 w_i) \tag{3}$$

```
class OjaNeuron:
    """
    Oja's rule: delta w = eta (x y - y^2 w)
    This prevents weight divergence by adding a normalization term
    """

    def __init__(self, n_inputs, learning_rate=0.01):
        self.lr = learning_rate
        self.weights = np.random.randn(n_inputs) * 0.1
        self.bias = 0  # Oja's rule typically doesn't use bias
        self.weight_history = []

    def activation(self, x):
        return np.dot(x, self.weights)

    def oja_update(self, x):
        """Oja's update rule"""
        y = self.activation(x)
        self.weights += self.lr * (y * x - y**2 * self.weights)
```

```
20
21      def train(self, X, epochs=50, verbose=True):
22          self.weight_history = []
23
24          for epoch in range(epochs):
25              for x in X:
26                  self.oja_update(x)
27
28              self.weight_history.append(self.weights.copy())
29
30              # Optional: normalize to unit length for stability
31              norm = np.linalg.norm(self.weights)
32              if norm > 0:
33                  self.weights = self.weights / norm
34
35              if verbose and (epoch + 1) % 10 == 0:
36                  print(f"Epoch {epoch+1}/{epochs}, Weights: {self.
                        weights}, Norm: {norm:.4f}")
37
38          return self
39
40  # Compare Hebb vs Oja on constant inputs
41  print("\n" + "="*60)
42  print("COMPARING HEBB VS OJA ON CONSTANT INPUTS")
43  print("="*60)
44
45  X_ones = np.ones((100, 2))
46
47  hebb_test = HebbianNeuron(2, learning_rate=0.01)
48  oja_test = OjaNeuron(2, learning_rate=0.01)
49
50  hebb_test.train(X_ones, epochs=100, verbose=False)
51  oja_test.train(X_ones, epochs=100, verbose=False)
52
53  plt.figure(figsize=(12, 4))
54
55  plt.subplot(1, 2, 1)
56  hebb_history = np.array(hebb_test.weight_history)
57  plt.plot(hebb_history[:, 0], label='w1 Hebb', linewidth=2)
58  plt.plot(hebb_history[:, 1], label='w2 Hebb', linewidth=2)
59  plt.title('Hebb Rule (diverging)')
60  plt.legend()
61  plt.grid(True, alpha=0.3)
62  plt.xlabel('Epoch')
63  plt.ylabel('Weight value')
64
65  plt.subplot(1, 2, 2)
66  oja_history = np.array(oja_test.weight_history)
67  plt.plot(oja_history[:, 0], label='w1 Oja', linewidth=2)
68  plt.plot(oja_history[:, 1], label='w2 Oja', linewidth=2)
69  plt.title('Oja Rule (stable)')
```

```
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlabel('Epoch')
plt.ylabel('Weight value')

plt.tight_layout()
plt.show()

print(f"Hebb final weights: {hebb_test.weights}")
print(f"Oja final weights: {oja_test.weights}")
print(f"Oja final weight norm: {np.linalg.norm(oja_test.weights):.4f}")
```

3.3.8 Hebbian Learning as Principal Component Analysis

Oja's rule actually performs Principal Component Analysis (PCA) - it finds the direction of maximum variance in the data:

```
# Generate data with a clear principal component
np.random.seed(42)
n_points = 300

# Create elongated cluster (strong principal component)
angle = np.pi / 4  # 45 degrees
rotation = np.array([[np.cos(angle), -np.sin(angle)],
                     [np.sin(angle), np.cos(angle)]])
X_elongated = np.random.randn(n_points, 2) @ rotation
X_elongated[:, 0] *= 3  # Stretch in first direction

# Train Oja neuron
oja_pca = OjaNeuron(2, learning_rate=0.01)
oja_pca.train(X_elongated, epochs=100, verbose=False)

# Visualize
plt.figure(figsize=(10, 8))
plt.scatter(X_elongated[:, 0], X_elongated[:, 1], alpha=0.5, label='Data')

# Plot Oja's learned direction
origin = np.array([0, 0])
scale = 3
plt.quiver(*origin, oja_pca.weights[0] * scale, oja_pca.weights[1] * scale,
           angles='xy', scale_units='xy', scale=1,
           color='red', label='Oja direction', width=0.02)

# Plot actual principal component (for comparison)
from sklearn.decomposition import PCA
pca = PCA(n_components=1)
pca.fit(X_elongated)
pca_direction = pca.components_[0]
```

```
plt.quiver(*origin, pca_direction[0] * scale, pca_direction[1] *
    scale,
           angles='xy', scale_units='xy', scale=1,
           color='green', label='PCA direction', width=0.02,
              alpha=0.7)

plt.xlabel('x1')
plt.ylabel('x2')
plt.title("Oja's Rule finds Principal Component")
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()

print(f"Oja direction: {oja_pca.weights}")
print(f"PCA direction: {pca_direction}")
print(f"Dot product (should be near 1): {np.abs(np.dot(oja_pca.
    weights, pca_direction)):.4f}")
```

3.3.9 Simple Associative Memory with Hebbian Learning

Hebbian learning can be used to create a simple associative memory:

```
class AssociativeMemory:
    """Simple associative memory using Hebbian learning"""

    def __init__(self, pattern_size, learning_rate=0.1):
        self.pattern_size = pattern_size
        self.lr = learning_rate
        # Initialize weights matrix (associative memory)
        self.weights = np.zeros((pattern_size, pattern_size))

    def memorize(self, pattern):
        """
        Memorize a pattern using Hebbian learning

        Parameters:
        pattern: array of -1/1 values
        """
        pattern = np.array(pattern)
        # Hebbian update: outer product
        self.weights += self.lr * np.outer(pattern, pattern)
        # Zero diagonal (no self-connection)
        np.fill_diagonal(self.weights, 0)

    def recall(self, partial_pattern, steps=5):
        """
        Recall a pattern from partial input

        Parameters:
```

```
        partial_pattern: array with -1/1 and possibly 0 for unknown
        steps: number of update steps

        Returns:
        array: recalled pattern
        """
        state = np.array(partial_pattern).copy()

        for _ in range(steps):
            for i in range(self.pattern_size):
                if partial_pattern[i] == 0:  # Only update unknown bits
                    # Compute net input
                    net_input = np.dot(self.weights[i], state)
                    state[i] = 1 if net_input > 0 else -1

        return state

    def print_pattern(self, pattern, title="Pattern"):
        """Print pattern as string for visualization (using ASCII)"""
        print(f"{title}:", end=" ")
        for p in pattern:
            if p == 1:
                print('1', end='')
            elif p == -1:
                print('0', end='')
            else:
                print('?', end='')
        print()

# Example: memorize and recall patterns
memory = AssociativeMemory(pattern_size=8)

# Create some patterns
pattern1 = np.array([1, 1, 1, -1, -1, -1, 1, 1])
pattern2 = np.array([1, -1, 1, -1, 1, -1, 1, -1])

print("="*60)
print("ASSOCIATIVE MEMORY WITH HEBBIAN LEARNING")
print("="*60)

memory.print_pattern(pattern1, "Pattern 1")
memory.print_pattern(pattern2, "Pattern 2")

# Memorize patterns
memory.memorize(pattern1)
memory.memorize(pattern2)

print("\n" + "-"*40)
```

```
print("Recall Tests")
print("-"*40)

# Test recall with partial patterns
partial1 = np.array([1, 1, 1, 0, 0, 0, 0, 0])  # Only first 3 bits known
recalled1 = memory.recall(partial1)

memory.print_pattern(partial1, "Partial input")
memory.print_pattern(recalled1, "Recalled    ")
print(f"Match pattern1: {np.all(recalled1 == pattern1)}")

partial2 = np.array([1, 0, 1, 0, 0, 0, 0, 0])  # Only first 3 bits with pattern
recalled2 = memory.recall(partial2)

memory.print_pattern(partial2, "Partial input")
memory.print_pattern(recalled2, "Recalled    ")
print(f"Match pattern2: {np.all(recalled2 == pattern2)}")

# Test with noisy pattern
noisy = pattern1.copy()
noisy[3] = 1  # Flip one bit
noisy[4] = 0  # Set to unknown
print(f"\nNoisy pattern:", end=" ")
for p in noisy:
    if p == 0:
        print('?', end='')
    elif p == 1:
        print('1', end='')
    else:
        print('0', end='')
print()

recalled_noisy = memory.recall(noisy)
memory.print_pattern(recalled_noisy, "Recovered   ")
print(f"Recovered original: {np.all(recalled_noisy == pattern1)}")
```

3.3.10 Visualizing Weight Evolution During Learning

```
def visualize_hebbian_learning_3d(X, learning_rate=0.01, epochs=50):
    """Visualize weight evolution in 3D for 3-input data"""
    from mpl_toolkits.mplot3d import Axes3D

    # Create 3D data if needed
    if X.shape[1] == 2:
        # Add a third dimension
```

```
        X_3d = np.column_stack([X, np.random.randn(X.shape[0]) *
            0.5])
    else:
        X_3d = X

    # Train Hebbian neuron
    hebb = HebbianNeuron(n_inputs=3, learning_rate=learning_rate)
    hebb.train(X_3d, epochs=epochs, verbose=False)

    # Create 3D plot
    fig = plt.figure(figsize=(12, 5))

    ax1 = fig.add_subplot(1, 2, 1, projection='3d')
    ax1.scatter(X_3d[:, 0], X_3d[:, 1], X_3d[:, 2], alpha=0.5, c=
        'blue')

    # Plot final weight vector
    ax1.quiver(0, 0, 0,
               hebb.weights[0], hebb.weights[1], hebb.weights[2],
               color='red', linewidth=3, arrow_length_ratio=0.1)

    ax1.set_xlabel('X1')
    ax1.set_ylabel('X2')
    ax1.set_zlabel('X3')
    ax1.set_title('Data and Final Weight Vector')

    ax2 = fig.add_subplot(1, 2, 2)
    weight_history = np.array(hebb.weight_history)
    ax2.plot(weight_history[:, 0], label='Weight 1', linewidth=2)
    ax2.plot(weight_history[:, 1], label='Weight 2', linewidth=2)
    ax2.plot(weight_history[:, 2], label='Weight 3', linewidth=2)
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Weight value')
    ax2.set_title('Weight Evolution')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    return hebb

# Generate 3D correlated data
np.random.seed(42)
X_3d = np.random.multivariate_normal(
    mean=[0, 0, 0],
    cov=[[1, 0.7, 0.5],
         [0.7, 1, 0.6],
         [0.5, 0.6, 1]],
    size=200
)
```

```
57
58 hebb_3d = visualize_hebbian_learning_3d(X_3d, learning_rate=0.01,
       epochs=50)
```

3.4 Summary of Hebbian Learning

Property	Description
Learning Type	Unsupervised
Update Rule	$\Delta w_i = \eta \times x_i \times y$
Activation	Usually sigmoid or linear
Advantages	Biologically plausible, local learning
Disadvantages	Weights can diverge, no error correction
Applications	Correlation detection, PCA, associative memory

Table 8: Summary of Hebbian Learning Properties

Key takeaways:

- Hebbian learning is **unsupervised** - it learns from input patterns only
- It discovers **correlations** in the input data
- Pure Hebbian learning is **unstable** - weights grow indefinitely
- **Oja's rule** provides a normalized, stable version
- Oja's rule performs **Principal Component Analysis** (PCA)
- Hebbian learning can create **associative memories**
- It cannot learn specific input-output mappings (needs supervision for that)

3.5 Additional Exercises

Exercise 2.11 **Hebbian learning with different activation functions:** Modify the Hebbian neuron to use tanh activation instead of sigmoid. Compare the results.

Exercise 2.12 **Decorrelation with anti-Hebbian learning:** Implement anti-Hebbian learning (negative learning rate) and observe its effect on correlated data.

Exercise 2.13 **Competitive learning:** Combine multiple Hebbian neurons to create a simple competitive learning network.

Exercise 2.14 **Hebbian learning for image patches:** Extract small patches from an image and use Hebbian learning to find recurring patterns.

Exercise 2.15 **Comparing Hebb, Oja, and PCA:** Generate datasets with different correlation structures and compare the directions learned by Hebb, Oja, and PCA.

4 Perceptron (Rosenblatt, 1957)

4.1 Historical Introduction

In 1957, psychologist Frank Rosenblatt, working at the Cornell Aeronautical Laboratory, proposed the Perceptron. This was a major breakthrough as it was the **first trainable neuron model** with supervised learning.

4.1.1 Context

- Rosenblatt was inspired by the work of McCulloch-Pitts and Hebb
- He wanted to create a machine capable of learning to recognize patterns
- The Perceptron was physically implemented in the Mark I Perceptron (1958)
- It was designed for image recognition with a 20x20 pixel retina

4.1.2 The Perceptron Model

Unlike McCulloch-Pitts, weights are not fixed in advance but **learned from examples**. The algorithm is simple:

1. Initialize weights randomly or to zero
2. For each training example (x, y_{target}):
   - Compute prediction: $y_{\text{pred}} = \text{sign}(\sum w_i x_i + b)$
   - If $y_{\text{pred}} \neq y_{\text{target}}$, update weights: $w_i \leftarrow w_i + \eta(y_{\text{target}} - y_{\text{pred}})x_i$
   - Update bias: $b \leftarrow b + \eta(y_{\text{target}} - y_{\text{pred}})$
3. Repeat until convergence (no more errors)

4.1.3 Mathematical Formulation

The Perceptron uses the sign activation function:

$$y_{\text{pred}} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 0 \\ -1 & \text{otherwise} \end{cases} \tag{4}$$

The update rule when there is a misclassification:

$$\Delta \mathbf{w} = \eta(y_{\text{target}} - y_{\text{pred}})\mathbf{x} \tag{5}$$

$$\Delta b = \eta(y_{\text{target}} - y_{\text{pred}}) \tag{6}$$

4.1.4 Properties

- **Supervised learning:** requires labeled training data
- **Convergence guaranteed** if data is linearly separable (Perceptron Convergence Theorem)
- **Activation function:** sign function (output -1 or 1)
- **Binary classifier:** separates two classes

4.1.5 Limitations

- Can only solve linearly separable problems
- Stops once separation is found (no margin optimization)
- No probability estimates, just hard decisions
- Sensitive to the order of training examples

4.2 Theoretical Exercises

Exercise 3.1 **Understanding the rule:** Explain the Perceptron update rule: why do we add $\eta(y_{\text{target}} - y_{\text{pred}})x_i$? What happens when the prediction is correct?

Exercise 3.2 **First learning step:** A Perceptron has initial weights [0.5, -0.2] and bias 0.1. Example (x=[1, -1], $y_{\text{target}} = 1$) is presented. Current prediction is -1. Calculate the update with eta=0.1. Show the new weights and bias.

Exercise 3.3 **Correct prediction case:** With the same initial weights, what happens if $y_{\text{pred}} = y_{\text{target}}$?

Exercise 3.4 **Learning AND gate:** Initialize a Perceptron with zero weights and zero bias. Apply the algorithm manually on the 4 examples of the AND gate (in -1/1 format). Show weights and bias after one complete epoch (all 4 examples). Use eta=0.1.

Exercise 3.5 **Learning OR gate:** Same exercise for the OR gate. Compare the final weights with those obtained for AND.

Exercise 3.6 **Convergence speed:** Approximately how many epochs are needed to learn AND with a Perceptron using eta=0.1?

Exercise 3.7 **Learning rate influence:** What happens if eta is too large? Too small? Give concrete examples.

Exercise 3.8 **Non-convergence on XOR:** Apply the Perceptron algorithm to the XOR problem. What do you observe? Will it ever converge?

Exercise 3.9 **Role of bias:** What is the role of bias in a Perceptron? What happens if bias is fixed to 0?

Exercise 3.10 **Margin:** The Perceptron stops as soon as all examples are correctly classified. Why is this not always optimal? What could be improved?

Exercise 3.11 **Perceptron Convergence Theorem:** State the Perceptron Convergence Theorem. What are its hypotheses? What does it guarantee?

Exercise 3.12 **Weight space interpretation:** In the weight space, what does each training example represent? How does the Perceptron update move the weight vector?

Exercise 3.13 **Multi-class Perceptron:** Propose an extension of the Perceptron for multi-class classification (one-vs-rest or one-vs-one).

Exercise 3.14 **Normalization:** Why is it important to normalize or standardize inputs for the Perceptron?

Exercise 3.15 **Voting Perceptron:** Explain the concept of the voting Perceptron. How does it differ from the standard Perceptron?

4.3 Implementation of Perceptron in Python

In this section, we will implement the Perceptron algorithm and explore its behavior on various datasets.

4.3.1 Basic Perceptron Implementation

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

class Perceptron:
    """Perceptron classifier"""

    def __init__(self, learning_rate=0.01, n_epochs=50,
       random_state=None):
        """
        Initialize Perceptron

        Parameters:
        learning_rate: eta - learning rate (default 0.01)
        n_epochs: number of passes over training data (default
           50)
        random_state: seed for random initialization (default
           None)
        """
        self.lr = learning_rate
        self.n_epochs = n_epochs
        self.random_state = random_state
        self.weights = None
        self.bias = None
        self.errors_history = []  # Track errors per epoch
        self.weights_history = []  # Track weights evolution

    def net_input(self, X):
        """Compute linear combination"""
        return np.dot(X, self.weights) + self.bias

    def predict(self, X):
        """Predict class labels (-1 or 1)"""
        return np.where(self.net_input(X) >= 0.0, 1, -1)

    def fit(self, X, y, verbose=True):
        """
        Train the Perceptron

        Parameters:
        X: training data of shape (n_samples, n_features)
        y: target values (-1 or 1) of shape (n_samples,)
```

```
        verbose: print progress

        Returns:
        self: trained Perceptron
        """
        n_samples, n_features = X.shape

        # Set random seed for reproducibility
        if self.random_state:
            np.random.seed(self.random_state)

        # Initialize weights and bias
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.errors_history = []
        self.weights_history = [self.weights.copy()]

        for epoch in range(self.n_epochs):
            errors = 0

            for xi, target in zip(X, y):
                # Compute prediction
                prediction = self.predict(xi.reshape(1, -1))[0]

                # Update if prediction is wrong
                if prediction != target:
                    update = self.lr * target
                    self.weights += update * xi
                    self.bias += update
                    errors += 1

            self.errors_history.append(errors)
            self.weights_history.append(self.weights.copy())

            if verbose and (epoch + 1) % 10 == 0:
                print(f"Epoch {epoch+1}/{self.n_epochs}, Errors:
                    {errors}")

            # Early stopping if no errors
            if errors == 0:
                if verbose:
                    print(f"Converged at epoch {epoch + 1}")
                break

        return self

    def score(self, X, y):
        """Compute accuracy score"""
        predictions = self.predict(X)
        return np.mean(predictions == y)

```

```python
# Test on simple data
X, y = make_classification(
    n_samples=200, n_features=2, n_redundant=0,
    n_clusters_per_class=1, class_sep=1.5, random_state=42
)
y = np.where(y == 0, -1, 1)  # Convert to -1/1

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

# Standardize (important for Perceptron)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Perceptron
perceptron = Perceptron(learning_rate=0.1, n_epochs=50,
    random_state=42)
perceptron.fit(X_train_scaled, y_train)

# Evaluate
train_acc = perceptron.score(X_train_scaled, y_train)
test_acc = perceptron.score(X_test_scaled, y_test)

print(f"\nTraining accuracy: {train_acc:.2%}")
print(f"Test accuracy: {test_acc:.2%}")
print(f"Final weights: {perceptron.weights}")
print(f"Final bias: {perceptron.bias:.4f}")
```

Listing 3: Basic Perceptron implementation

4.3.2 Learning Logic Gates with Perceptron

Let's train the Perceptron on logic gates:

```python
# Data for logic gates (in -1/1 format)
X_logic = np.array([[0,0], [0,1], [1,0], [1,1]])

# Truth tables in -1/1 format (-1 for 0, 1 for 1)
y_and = np.array([-1, -1, -1, 1])    # AND: only (1,1) is positive
y_or = np.array([-1, 1, 1, 1])       # OR: only (0,0) is negative
y_nand = np.array([1, 1, 1, -1])     # NAND: only (1,1) is
    negative
y_nor = np.array([1, -1, -1, -1])    # NOR: only (0,0) is
    positive
y_xor = np.array([-1, 1, 1, -1])     # XOR: impossible with
    single neuron

def train_and_evaluate_perceptron(X, y, gate_name, learning_rate
    =0.1, epochs=20):
    """Train Perceptron on logic gate and evaluate"""
```

```
13
14      print(f"\n{'='*50}")
15      print(f"Training Perceptron on {gate_name} gate")
16      print('='*50)
17
18      perceptron = Perceptron(learning_rate=learning_rate, n_epochs
            =epochs, random_state=42)
19      perceptron.fit(X, y, verbose=False)
20
21      predictions = perceptron.predict(X)
22
23      print(f"Weights: {perceptron.weights}")
24      print(f"Bias: {perceptron.bias:.4f}")
25      print("\nTruth Table:")
26      print("A  B  | Target | Prediction | Status")
27      print("----+--------+------------+-------")
28      for i, (x, target, pred) in enumerate(zip(X, y, predictions))
            :
29          status = "OK" if target == pred else "FAIL"
30          target_val = 1 if target == 1 else 0
31          pred_val = 1 if pred == 1 else 0
32          print(f"{int(x[0])}  {int(x[1])}  |   {target_val}    |
                    {pred_val}     |  {status}")
33
34      accuracy = np.mean(predictions == y)
35      print(f"\nAccuracy: {accuracy:.2%}")
36      print(f"Epochs to converge: {len(perceptron.errors_history)}"
            )
37
38      # Plot error evolution
39      plt.figure(figsize=(10, 4))
40      plt.subplot(1, 2, 1)
41      plt.plot(perceptron.errors_history, marker='o', linestyle='-'
            , linewidth=2)
42      plt.xlabel('Epoch')
43      plt.ylabel('Number of errors')
44      plt.title(f'{gate_name} Gate - Error Evolution')
45      plt.grid(True, alpha=0.3)
46      plt.ylim(-0.5, 4.5)
47
48      # Plot weight evolution
49      plt.subplot(1, 2, 2)
50      weights_history = np.array(perceptron.weights_history)
51      plt.plot(weights_history[:, 0], label='Weight 1', linewidth
            =2)
52      plt.plot(weights_history[:, 1], label='Weight 2', linewidth
            =2)
53      plt.xlabel('Update step')
54      plt.ylabel('Weight value')
55      plt.title(f'{gate_name} Gate - Weight Evolution')
56      plt.legend()
```

```
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    return perceptron

# Train on different gates
p_and = train_and_evaluate_perceptron(X_logic, y_and, "AND")
p_or = train_and_evaluate_perceptron(X_logic, y_or, "OR")
p_nand = train_and_evaluate_perceptron(X_logic, y_nand, "NAND")
p_nor = train_and_evaluate_perceptron(X_logic, y_nor, "NOR")
p_xor = train_and_evaluate_perceptron(X_logic, y_xor, "XOR")
```

4.3.3 Observations on XOR

The XOR gate will not converge. Let's analyze why:

```
# Detailed analysis of XOR failure
print("\n" + "="*60)
print("ANALYSIS OF XOR FAILURE")
print("="*60)

X_xor = np.array([[0,0], [0,1], [1,0], [1,1]])
y_xor = np.array([-1, 1, 1, -1])

# Try with many epochs
p_xor_detailed = Perceptron(learning_rate=0.1, n_epochs=100,
    random_state=42)
p_xor_detailed.fit(X_xor, y_xor, verbose=False)

print(f"Final weights after 100 epochs: {p_xor_detailed.weights}"
    )
print(f"Final bias: {p_xor_detailed.bias:.4f}")
print(f"Final errors: {p_xor_detailed.errors_history[-1]}/4")

# Show that weights oscillate
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(p_xor_detailed.errors_history, linewidth=2, color='red')
plt.xlabel('Epoch')
plt.ylabel('Number of errors')
plt.title('XOR - Errors never reach zero')
plt.grid(True, alpha=0.3)
plt.ylim(-0.5, 4.5)

plt.subplot(1, 2, 2)
weights_history = np.array(p_xor_detailed.weights_history)
plt.plot(weights_history[:, 0], label='Weight 1', linewidth=2)
plt.plot(weights_history[:, 1], label='Weight 2', linewidth=2)
```

```
plt.xlabel('Update step')
plt.ylabel('Weight value')
plt.title('XOR - Weights oscillate')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Geometric explanation
plt.figure(figsize=(8, 8))
colors = ['blue' if y == -1 else 'red' for y in y_xor]
plt.scatter(X_xor[:, 0], X_xor[:, 1], c=colors, s=200, edgecolors='black', linewidth=2)

# Try to plot decision boundary from final weights
x_line = np.linspace(-0.5, 1.5, 100)
if p_xor_detailed.weights[1] != 0:
    y_line = -(p_xor_detailed.weights[0] * x_line + p_xor_detailed.bias) / p_xor_detailed.weights[1]
    plt.plot(x_line, y_line, 'g--', linewidth=2, label='Attempted boundary')

plt.xlabel('x1')
plt.ylabel('x2')
plt.title('XOR Problem - Not Linearly Separable')
plt.grid(True, alpha=0.3)
plt.xlim(-0.2, 1.2)
plt.ylim(-0.2, 1.2)
plt.legend()
plt.show()

print("\n" + "="*60)
print("CONCLUSION: XOR is not linearly separable")
print("="*60)
print("No single straight line can separate the red and blue points.")
print("This is why the Perceptron cannot learn XOR.")
print("A multi-layer network is needed for such problems.")
```

4.3.4 Effect of Learning Rate

Let's explore how the learning rate affects convergence:

```
def compare_perceptron_learning_rates(X, y, learning_rates=[0.001, 0.01, 0.1, 1.0, 10.0], epochs=50):
    """
    Compare Perceptron performance with different learning rates
    """
    plt.figure(figsize=(15, 12))

```

```
    results = {}

    for i, lr in enumerate(learning_rates):
        # Train Perceptron
        p = Perceptron(learning_rate=lr, n_epochs=epochs,
            random_state=42)
        p.fit(X, y, verbose=False)

        # Plot error evolution
        plt.subplot(3, 2, i+1)
        plt.plot(p.errors_history, linewidth=2, marker='o',
            markersize=4)
        plt.xlabel('Epoch')
        plt.ylabel('Number of errors')
        plt.title(f'Learning Rate = {lr}')
        plt.grid(True, alpha=0.3)
        plt.ylim(-0.5, max(4, max(p.errors_history)+0.5))

        # Store results
        final_errors = p.errors_history[-1] if p.errors_history
            else 0
        epochs_to_converge = len(p.errors_history) if
            final_errors == 0 else epochs

        results[lr] = {
            'final_weights': p.weights.copy(),
            'bias': p.bias,
            'final_errors': final_errors,
            'epochs_to_converge': epochs_to_converge,
            'weights_history': np.array(p.weights_history)
        }

    plt.tight_layout()
    plt.show()

    # Print comparison
    print("\n" + "="*70)
    print("LEARNING RATE COMPARISON")
    print("="*70)
    print(f"{'LR':<10} {'Final Weights':<25} {'Bias':<10} {'
        Epochs':<10} {'Final Errors':<12}")
    print("-"*70)

    for lr, res in results.items():
        weights_str = f"[{res['final_weights'][0]:.3f}, {res['
            final_weights'][1]:.3f}]"
        print(f"{lr:<10.4f} {weights_str:<25} {res['bias']:<10.3f
            } "
              f"{res['epochs_to_converge']:<10} {res['
                  final_errors']:<12}")

```

```
    return results

# Test on AND gate
print("\n" + "="*70)
print("TESTING DIFFERENT LEARNING RATES ON AND GATE")
print("="*70)
results_and = compare_perceptron_learning_rates(X_logic, y_and)

# Test on a more complex dataset
X_comp, y_comp = make_classification(
    n_samples=200, n_features=2, n_redundant=0,
    n_clusters_per_class=1, class_sep=1.0, random_state=42
)
y_comp = np.where(y_comp == 0, -1, 1)

scaler = StandardScaler()
X_comp_scaled = scaler.fit_transform(X_comp)

print("\n" + "="*70)
print("TESTING DIFFERENT LEARNING RATES ON COMPLEX DATA")
print("="*70)
results_comp = compare_perceptron_learning_rates(X_comp_scaled,
   y_comp, epochs=100)
```

4.3.5 Decision Boundary Visualization

Let's visualize the decision boundary learned by the Perceptron:

```
def plot_perceptron_decision_boundary(perceptron, X, y, title="
   Perceptron Decision Boundary"):
    """
    Plot the decision boundary of a trained Perceptron
    """
    # Create mesh grid
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200),
        np.linspace(y_min, y_max, 200)
    )

    # Predict on grid
    Z = perceptron.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot
    plt.figure(figsize=(10, 8))
    plt.contourf(xx, yy, Z, alpha=0.3, levels=[-1, 0, 1], colors
       =['blue', 'red'])
    plt.contour(xx, yy, Z, levels=[0], colors='black', linewidths
       =2)
```

```
21
22      # Plot data points
23      scatter = plt.scatter(X[:, 0], X[:, 1], c=y,
24                            cmap='bwr', edgecolors='black', s=100,
                                 alpha=0.8)
25
26      # Plot weight vector (direction of decision boundary)
27      if perceptron.weights is not None:
28          # The decision boundary is perpendicular to the weight
                vector
29          w = perceptron.weights
30          b = perceptron.bias
31
32          # Plot weight vector
33          origin = np.array([x_min + 0.5, y_min + 0.5])
34          plt.quiver(origin[0], origin[1], w[0], w[1],
35                     angles='xy', scale_units='xy', scale=3,
36                     color='green', label='Weight vector', width
                          =0.02)
37
38          # Add text for weights and bias
39          plt.text(x_min + 0.5, y_max - 0.5,
40                   f'w = [{w[0]:.2f}, {w[1]:.2f}]\nb = {b:.2f}',
41                   bbox=dict(boxstyle="round,pad=0.3", facecolor="
                        yellow", alpha=0.7))
42
43      plt.xlabel('Feature 1')
44      plt.ylabel('Feature 2')
45      plt.title(title)
46      plt.colorbar(scatter)
47      plt.grid(True, alpha=0.3)
48      plt.legend()
49      plt.show()
50
51  # Generate and plot for a simple dataset
52  X_viz, y_viz = make_classification(
53      n_samples=100, n_features=2, n_redundant=0,
54      n_clusters_per_class=1, class_sep=2.0, random_state=42
55  )
56  y_viz = np.where(y_viz == 0, -1, 1)
57
58  scaler = StandardScaler()
59  X_viz_scaled = scaler.fit_transform(X_viz)
60
61  p_viz = Perceptron(learning_rate=0.1, n_epochs=50, random_state
        =42)
62  p_viz.fit(X_viz_scaled, y_viz, verbose=False)
63
64  plot_perceptron_decision_boundary(p_viz, X_viz_scaled, y_viz,
65                                    "Perceptron Decision Boundary")
66
```

```
# Plot for AND gate
p_and_viz = Perceptron(learning_rate=0.1, n_epochs=20,
    random_state=42)
p_and_viz.fit(X_logic, y_and, verbose=False)
plot_perceptron_decision_boundary(p_and_viz, X_logic, y_and,
                                  "AND Gate - Perceptron Boundary"
                                    )
```

4.3.6 Effect of Data Ordering

The Perceptron is sensitive to the order of training examples:

```
def demonstrate_order_sensitivity(X, y, n_experiments=5):
    """
    Demonstrate that Perceptron convergence depends on example
       order
    """
    print("\n" + "="*60)
    print("EFFECT OF DATA ORDERING ON PERCEPTRON")
    print("="*60)

    results = []

    for exp in range(n_experiments):
        # Shuffle data differently each time
        indices = np.random.permutation(len(X))
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        # Train Perceptron
        p = Perceptron(learning_rate=0.1, n_epochs=50,
            random_state=None)
        p.fit(X_shuffled, y_shuffled, verbose=False)

        epochs = len(p.errors_history)
        final_weights = p.weights.copy()

        results.append({
            'experiment': exp + 1,
            'epochs': epochs,
            'weights': final_weights
        })

        print(f"Experiment {exp+1}: Converged in {epochs} epochs,
            "
              f"Weights = [{final_weights[0]:.3f}, {final_weights
                  [1]:.3f}]")

    # Plot differences
    plt.figure(figsize=(10, 5))

```

```
    for i, res in enumerate(results):
        plt.scatter(res['weights'][0], res['weights'][1],
                    s=100, label=f'Exp {i+1}', alpha=0.7)

    plt.xlabel('Weight 1')
    plt.ylabel('Weight 2')
    plt.title('Different final weights due to ordering')
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()

    return results

# Test on AND gate
print("\nAND Gate - Different initial orders:")
order_results = demonstrate_order_sensitivity(X_logic, y_and,
   n_experiments=5)
```

4.3.7 Voting Perceptron

An improved version that keeps track of all weight vectors:

```
class VotingPerceptron:
    """
    Voting Perceptron - keeps track of all weight vectors
    and votes based on their survival time
    """

    def __init__(self, learning_rate=0.01, n_epochs=50):
        self.lr = learning_rate
        self.n_epochs = n_epochs
        self.weight_vectors = []  # List of (weights, bias, count
           )
        self.current_weights = None
        self.current_bias = None
        self.current_count = 0

    def predict(self, X):
        """Predict using voting"""
        if not self.weight_vectors:
            return np.zeros(len(X))

        predictions = np.zeros(len(X))

        for weights, bias, count in self.weight_vectors:
            pred = np.where(np.dot(X, weights) + bias >= 0, 1,
               -1)
            predictions += count * pred

        return np.where(predictions >= 0, 1, -1)

```

```
    def fit(self, X, y):
        n_samples, n_features = X.shape

        self.current_weights = np.zeros(n_features)
        self.current_bias = 0
        self.current_count = 1
        self.weight_vectors = []

        for epoch in range(self.n_epochs):
            for xi, target in zip(X, y):
                prediction = np.where(np.dot(xi, self.
                    current_weights) + self.current_bias >= 0, 1,
                    -1)

                if prediction != target:
                    # Save current vector with its count
                    self.weight_vectors.append(
                        (self.current_weights.copy(), self.
                            current_bias, self.current_count)
                    )

                    # Update
                    update = self.lr * target
                    self.current_weights += update * xi
                    self.current_bias += update
                    self.current_count = 1
                else:
                    self.current_count += 1

        # Add final vector
        self.weight_vectors.append(
            (self.current_weights.copy(), self.current_bias, self
                .current_count)
        )

        return self

    def score(self, X, y):
        predictions = self.predict(X)
        return np.mean(predictions == y)

# Compare standard vs voting Perceptron
X_comp, y_comp = make_classification(
    n_samples=200, n_features=2, n_redundant=0,
    n_clusters_per_class=1, class_sep=1.2, random_state=42
)
y_comp = np.where(y_comp == 0, -1, 1)

X_train, X_test, y_train, y_test = train_test_split(X_comp,
    y_comp, test_size=0.3, random_state=42)

```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Standard Perceptron
std_p = Perceptron(learning_rate=0.1, n_epochs=50, random_state=42)
std_p.fit(X_train_scaled, y_train, verbose=False)
std_test_acc = std_p.score(X_test_scaled, y_test)

# Voting Perceptron
voting_p = VotingPerceptron(learning_rate=0.1, n_epochs=50)
voting_p.fit(X_train_scaled, y_train)
voting_test_acc = voting_p.score(X_test_scaled, y_test)

print("\n" + "="*60)
print("STANDARD VS VOTING PERCEPTRON")
print("="*60)
print(f"Standard Perceptron - Test accuracy: {std_test_acc:.2%}")
print(f"Voting Perceptron - Test accuracy: {voting_test_acc:.2%}")
print(f"Number of weight vectors saved: {len(voting_p.weight_vectors)}")
```

4.3.8 Perceptron on Real Dataset: Iris

Let's apply the Perceptron to the Iris dataset:

```
from sklearn.datasets import load_iris
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Load Iris dataset (2 classes only)
iris = load_iris()
X_iris = iris.data[:100, :2]  # First 2 classes, first 2 features
y_iris = iris.target[:100]
y_iris = np.where(y_iris == 0, -1, 1)  # Convert to -1/1

# Split and standardize
X_train, X_test, y_train, y_test = train_test_split(
    X_iris, y_iris, test_size=0.3, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Perceptron with different learning rates
learning_rates = [0.001, 0.01, 0.1, 1.0]
best_acc = 0
```

```
best_lr = None
best_p = None

plt.figure(figsize=(15, 10))

for i, lr in enumerate(learning_rates):
    p = Perceptron(learning_rate=lr, n_epochs=100, random_state=42)
    p.fit(X_train_scaled, y_train, verbose=False)

    train_acc = p.score(X_train_scaled, y_train)
    test_acc = p.score(X_test_scaled, y_test)

    plt.subplot(2, 2, i+1)
    plt.plot(p.errors_history, linewidth=2, marker='o', markersize=3)
    plt.xlabel('Epoch')
    plt.ylabel('Number of errors')
    plt.title(f'LR={lr} - Train: {train_acc:.2%}, Test: {test_acc:.2%}')
    plt.grid(True, alpha=0.3)

    if test_acc > best_acc:
        best_acc = test_acc
        best_lr = lr
        best_p = p

plt.tight_layout()
plt.show()

print("\n" + "="*60)
print("PERCEPTRON ON IRIS DATASET")
print("="*60)
print(f"Best learning rate: {best_lr}")
print(f"Best test accuracy: {best_acc:.2%}")
print(f"Final weights: {best_p.weights}")
print(f"Final bias: {best_p.bias:.4f}")

# Confusion matrix
test_pred = best_p.predict(X_test_scaled)
cm = confusion_matrix(y_test, test_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Setosa', 'Versicolor'],
            yticklabels=['Setosa', 'Versicolor'])
plt.title('Confusion Matrix - Perceptron on Iris')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

```

```
# Decision boundary
plot_perceptron_decision_boundary(best_p, X_test_scaled, y_test,
                                  f"Perceptron on Iris (LR={
                                     best_lr}, Acc={best_acc:.2%})"
                                     )
```

4.3.9 Perceptron with Non-Separable Data

What happens when data is not linearly separable?

```
# Generate non-linearly separable data (circles)
from sklearn.datasets import make_circles

X_circles, y_circles = make_circles(n_samples=300, factor=0.5,
    noise=0.1, random_state=42)
y_circles = np.where(y_circles == 0, -1, 1)

# Split
X_train, X_test, y_train, y_test = train_test_split(X_circles,
    y_circles, test_size=0.3, random_state=42)

# Standardize
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Perceptron
p_circles = Perceptron(learning_rate=0.1, n_epochs=200,
    random_state=42)
p_circles.fit(X_train_scaled, y_train, verbose=False)

# Evaluate
train_acc = p_circles.score(X_train_scaled, y_train)
test_acc = p_circles.score(X_test_scaled, y_test)

print("\n" + "="*60)
print("PERCEPTRON ON NON-SEPARABLE DATA (CIRCLES)")
print("="*60)
print(f"Training accuracy: {train_acc:.2%}")
print(f"Test accuracy: {test_acc:.2%}")
print(f"Final errors: {p_circles.errors_history[-1]}/{len(y_train
    )}")

# Plot results
plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.scatter(X_train_scaled[:, 0], X_train_scaled[:, 1], c=y_train
    ,
            cmap='bwr', edgecolors='black', alpha=0.7)
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')
plt.title('Training Data (Circles)')
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 2)
plt.plot(p_circles.errors_history, linewidth=2, color='red')
plt.xlabel('Epoch')
plt.ylabel('Number of errors')
plt.title('Error Evolution (never converges)')
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 3)
# Plot decision boundary (even though it's not meaningful)
plot_perceptron_decision_boundary(p_circles, X_test_scaled,
    y_test,
                                  "Attempted Boundary (fails)")

plt.tight_layout()
plt.show()

print("\nCONCLUSION: The Perceptron cannot separate non-linearly
    separable data.")
print("The error never reaches zero, and the decision boundary is
     inadequate.")
```

4.3.10 Comparison with Scikit-Learn Perceptron

```
from sklearn.linear_model import Perceptron as SklearnPerceptron
from sklearn.metrics import accuracy_score

# Generate data
X_sk, y_sk = make_classification(
    n_samples=300, n_features=2, n_redundant=0,
    n_clusters_per_class=1, class_sep=1.5, random_state=42
)
y_sk = np.where(y_sk == 0, -1, 1)

X_train, X_test, y_train, y_test = train_test_split(X_sk, y_sk,
    test_size=0.3, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Our Perceptron
my_p = Perceptron(learning_rate=0.1, n_epochs=50, random_state
    =42)
my_p.fit(X_train_scaled, y_train, verbose=False)
my_pred = my_p.predict(X_test_scaled)
my_acc = accuracy_score(y_test, my_pred)
```

```python

# sklearn Perceptron
sk_p = SklearnPerceptron(eta0=0.1, max_iter=50, random_state=42,
    shuffle=True)
sk_p.fit(X_train_scaled, y_train)
sk_pred = sk_p.predict(X_test_scaled)
sk_acc = accuracy_score(y_test, sk_pred)

print("\n" + "="*60)
print("COMPARISON WITH SCIKIT-LEARN PERCEPTRON")
print("="*60)
print(f"Our Perceptron - Test accuracy: {my_acc:.2%}")
print(f"sklearn Perceptron - Test accuracy: {sk_acc:.2%}")
print(f"\nOur weights: {my_p.weights}")
print(f"sklearn weights: {sk_p.coef_[0]}")
print(f"Our bias: {my_p.bias:.4f}")
print(f"sklearn bias: {sk_p.intercept_[0]:.4f}")

# Visualize both decision boundaries
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plot_perceptron_decision_boundary(my_p, X_test_scaled, y_test,
                                  "Our Perceptron")

plt.subplot(1, 2, 2)
# For sklearn, we need to create a wrapper
class SklearnPerceptronWrapper:
    def __init__(self, model):
        self.model = model
        self.weights = model.coef_[0]
        self.bias = model.intercept_[0]

    def predict(self, X):
        return self.model.predict(X)

sk_wrapper = SklearnPerceptronWrapper(sk_p)
plot_perceptron_decision_boundary(sk_wrapper, X_test_scaled,
    y_test,
                                  "sklearn Perceptron")

plt.tight_layout()
plt.show()
```

4.4 Summary of Perceptron

Key takeaways:

- The Perceptron is the **first trainable neural network** for classification
- It uses **supervised learning** with a simple error-driven update

Property	Description
Learning Type	Supervised
Activation Function	Sign (step) function
Update Rule	$\Delta w = \eta(y_{\text{target}} - y_{\text{pred}})x$
Convergence	Guaranteed for linearly separable data
Advantages	Simple, fast, guaranteed convergence
Disadvantages	Only linear separation, no probabilities, order sensitive
Applications	Binary classification, linear separability checking

Table 9: Summary of Perceptron Properties

- The **Perceptron Convergence Theorem** guarantees convergence if data is linearly separable
- It can learn AND, OR, NAND, NOR but **cannot learn XOR**
- The decision boundary is a **hyperplane** in feature space
- Performance depends on **learning rate** and **data ordering**
- The **Voting Perceptron** improves stability by averaging multiple solutions
- Despite its limitations, it remains an important educational tool and baseline

4.5 Additional Exercises

Exercise 3.16 **Averaged Perceptron:** Implement the averaged Perceptron which uses the average of all weight vectors seen during training. Compare with standard and voting Perceptron.

Exercise 3.17 **Kernel Perceptron:** Research and implement the kernel Perceptron for non-linear problems. Test on the circles dataset.

Exercise 3.18 **Online vs Batch Learning:** Modify the Perceptron to work in batch mode (update after all examples). Compare convergence speed.

Exercise 3.19 **Weight Initialization:** Experiment with different weight initialization strategies (zeros, small random, large random). How does it affect convergence?

Exercise 3.20 **Perceptron with Margin:** Implement a version that only updates when the margin is less than a threshold. Does this improve generalization?

Exercise 3.21 **Multi-class Perceptron:** Extend the Perceptron to handle more than 2 classes using one-vs-rest strategy.

Exercise 3.22 **Perceptron for Regression:** Can the Perceptron be adapted for regression? Propose a modification.

Exercise 3.23 **Visualizing the Weight Space:** For 2D data, plot the trajectory of the weight vector during training. What patterns do you observe?

Exercise 3.24 **Noisy Data:** Add label noise to the training data and observe how the Perceptron behaves. Does it still converge?

Exercise 3.25 **Feature Engineering:** Create polynomial features to make XOR linearly separable. Train a Perceptron on these features.

5 ADALINE (Widrow & Hoff, 1960)

5.1 Historical Introduction

In 1960, Bernard Widrow and his student Ted Hoff (who later co-founded Intel) developed ADALINE (ADAptive LInear NEuron) at Stanford University. This was a fundamental improvement over the Perceptron and laid the groundwork for modern adaptive signal processing.

5.1.1 Context

- Widrow and Hoff were working on pattern recognition and adaptive systems
- They wanted to create a neuron that could learn more smoothly than the Perceptron
- ADALINE was the first neural network applied to real-world problems (echo cancellation in telephone lines)
- The LMS (Least Mean Squares) algorithm became widely used in signal processing

5.1.2 Major Innovations

1. **Linear activation during learning:** Instead of the sign function, ADALINE uses the identity function during learning (output = net input). This makes the cost function differentiable.
2. **Mean Square Error minimization:** We minimize the sum of squared errors between desired output and linear output:

$$E = \frac{1}{2}\sum_{i=1}^{n}(y_{\text{target}}^{(i)} - y_{\text{linear}}^{(i)})^2 \tag{7}$$

3. **Gradient descent:** Weights are updated in the opposite direction of the error gradient:

$$\Delta \mathbf{w} = \eta(y_{\text{target}} - y_{\text{linear}})\mathbf{x} \tag{8}$$

$$\Delta b = \eta(y_{\text{target}} - y_{\text{linear}}) \tag{9}$$

4. **LMS rule (Least Mean Squares):** Also called the Widrow-Hoff rule, it minimizes the mean square error.

5.1.3 Key Difference from Perceptron

- **Perceptron:** updates weights only when there's a classification error, uses sign function
- **ADALINE:** updates weights for every example, proportionally to the error, uses linear output during training
- ADALINE's cost function is **convex**, guaranteeing convergence to global minimum
- ADALINE provides a **continuous error measure**, not just binary correctness

5.1.4 Mathematical Formulation

During training, ADALINE uses linear activation:

$$y_{\text{linear}} = \mathbf{w} \cdot \mathbf{x} + b \tag{10}$$

The cost function to minimize:

$$J(\mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^{n} (y_{\text{target}}^{(i)} - y_{\text{linear}}^{(i)})^2 \tag{11}$$

The gradient descent update:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{i=1}^{n} (y_{\text{target}}^{(i)} - y_{\text{linear}}^{(i)}) \mathbf{x}^{(i)} \tag{12}$$

For stochastic gradient descent (online learning):

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y_{\text{target}}^{(i)} - y_{\text{linear}}^{(i)}) \mathbf{x}^{(i)} \tag{13}$$

For prediction (classification), we apply the sign function:

$$y_{\text{pred}} = \text{sign}(y_{\text{linear}}) = \begin{cases} +1 & \text{if } y_{\text{linear}} \geq 0 \\ -1 & \text{otherwise} \end{cases} \tag{14}$$

5.2 Theoretical Exercises

Exercise 4.1 **Perceptron/ADALINE comparison:** Explain the fundamental difference between Perceptron and ADALINE learning rules. Why is ADALINE's approach potentially more stable?

Exercise 4.2 **Gradient calculation:** Derive the ADALINE update rule from the cost function $E = \frac{1}{2}(y_{\text{target}} - y_{\text{linear}})^2$. Show all steps.

Exercise 4.3 **First ADALINE step:** An ADALINE neuron has weights [0.2, -0.1] and bias 0.05. Example (x=[1, -1], $y_{\text{target}} = 1$) is presented. Linear output is 0.35. Calculate the update with eta=0.1. Show new weights and bias.

Exercise 4.4 **Update comparison:** Revisit Exercise 3.2 (Perceptron) and compare with the ADALINE update. What do you notice about the magnitude of updates?

Exercise 4.5 **Error influence:** In ADALINE, if the error is large, the update is large. Why is this desirable? How does this differ from Perceptron?

Exercise 4.6 **Cost function shape:** Sketch the MSE cost function for a 1-weight problem. Why is it convex? What does this imply about convergence?

Exercise 4.7 **Gradient descent explanation:** Explain the principle of gradient descent in your own words. Use the analogy of walking downhill in fog.

Exercise 4.8 **Optimal learning rate:** What happens if eta is too large in ADALINE? Too small? Give mathematical intuition.

Exercise 4.9 **Standardization importance:** Why is data standardization particularly important for ADALINE? Show with an example of features with different scales.

Exercise 4.10 **ADALINE for regression:** Can ADALINE be used for regression? How would you modify it? What would be the output?

Exercise 4.11 **Two-phase operation:** During learning, ADALINE uses linear output. For final prediction, it uses the sign function. Why these two phases? What would happen if we used sign during training?

Exercise 4.12 **Global minimum:** Why does gradient descent find the global minimum for ADALINE? Is this always true for all neural networks?

Exercise 4.13 **Stopping criterion:** Propose a stopping criterion for ADALINE learning other than fixed epochs. How would you detect convergence?

Exercise 4.14 **ADALINE and noise:** How does ADALINE behave with noisy data? Compare with Perceptron.

Exercise 4.15 **LMS vs Batch GD:** What's the difference between the LMS rule (incremental update) and batch gradient descent? When would you use each?

Exercise 4.16 **Learning rate decay:** Propose a learning rate decay schedule for ADALINE. Why might this be beneficial?

Exercise 4.17 **Momentum:** How could you add momentum to ADALINE updates? Write the modified update rule.

Exercise 4.18 **ADALINE for online learning:** Explain how ADALINE can be used for online learning where data arrives sequentially.

Exercise 4.19 **Connection to linear regression:** How is ADALINE related to linear regression? What's the difference?

Exercise 4.20 **Weight initialization:** Compare different weight initialization strategies for ADALINE (zeros, small random, large random). Which is best and why?

5.3 Implementation of ADALINE in Python

In this section, we will implement ADALINE and explore its behavior on various datasets.

5.3.1 Basic ADALINE Implementation

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_regression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

class Adaline:
    """ADAptive LInear NEuron classifier"""

    def __init__(self, learning_rate=0.01, n_epochs=50,
                 batch_size=None, random_state=None):
        """
        Initialize ADALINE

        Parameters:
        learning_rate: eta - learning rate (default 0.01)
        n_epochs: number of passes over training data (default
            50)
        batch_size: None for SGD, 1 for online, >1 for mini-batch
            , 'batch' for batch GD
        random_state: seed for random initialization
        """
        self.lr = learning_rate
        self.n_epochs = n_epochs
        self.batch_size = batch_size
        self.random_state = random_state
        self.weights = None
        self.bias = None
        self.cost_history = []  # Track MSE per epoch
        self.weights_history = []  # Track weights evolution

    def net_input(self, X):
        """Compute linear combination"""
        return np.dot(X, self.weights) + self.bias

    def activation(self, X):
        """
        Linear activation function (identity)
        Used during training
        """
        return self.net_input(X)

    def predict(self, X):
        """
```

```
43          Prediction for classification (after training)
44          Returns -1 or 1
45          """
46          return np.where(self.net_input(X) >= 0.0, 1, -1)
47
48      def _initialize_weights(self, n_features):
49          """Initialize weights and bias"""
50          if self.random_state:
51              np.random.seed(self.random_state)
52
53          # Small random initialization works better than zeros
54          self.weights = np.random.randn(n_features) * 0.01
55          self.bias = 0.0
56
57      def fit(self, X, y, verbose=True):
58          """
59          Train ADALINE using gradient descent
60
61          Parameters:
62          X: training data of shape (n_samples, n_features)
63          y: target values (-1 or 1) of shape (n_samples,)
64          verbose: print progress
65
66          Returns:
67          self: trained ADALINE
68          """
69          n_samples, n_features = X.shape
70
71          self._initialize_weights(n_features)
72          self.cost_history = []
73          self.weights_history = [self.weights.copy()]
74
75          # Determine batch processing mode
76          if self.batch_size == 'batch':
77              # Batch gradient descent (use all samples)
78              batch_indices = [np.arange(n_samples)]
79          elif self.batch_size is None or self.batch_size >=
               n_samples:
80              # Stochastic gradient descent (one sample at a time)
81              batch_indices = [[i] for i in range(n_samples)]
82          else:
83              # Mini-batch gradient descent
84              n_batches = max(1, n_samples // self.batch_size)
85              batch_indices = []
86              for i in range(n_batches):
87                  start = i * self.batch_size
88                  end = min((i + 1) * self.batch_size, n_samples)
89                  batch_indices.append(np.arange(start, end))
90
91          for epoch in range(self.n_epochs):
92              # Shuffle data for stochastic/mini-batch
```

```
            if self.batch_size != 'batch':
                indices = np.random.permutation(n_samples)
                X_shuffled = X[indices]
                y_shuffled = y[indices]
            else:
                X_shuffled = X
                y_shuffled = y

            epoch_cost = 0

            for batch_idx in batch_indices:
                X_batch = X_shuffled[batch_idx]
                y_batch = y_shuffled[batch_idx]

                # Compute linear output for batch
                output = self.activation(X_batch)

                # Compute errors
                errors = y_batch - output

                # Update weights (gradient descent)
                self.weights += self.lr * np.dot(X_batch.T,
                    errors) / len(batch_idx)
                self.bias += self.lr * np.sum(errors) / len(
                    batch_idx)

                # Accumulate cost
                epoch_cost += np.sum(errors**2)

            # Average cost for this epoch
            epoch_cost /= (2 * n_samples)
            self.cost_history.append(epoch_cost)
            self.weights_history.append(self.weights.copy())

            if verbose and (epoch + 1) % 10 == 0:
                print(f"Epoch {epoch+1}/{self.n_epochs}, Cost: {
                    epoch_cost:.6f}")

        return self

    def score(self, X, y):
        """Compute accuracy for classification"""
        predictions = self.predict(X)
        return np.mean(predictions == y)

# Test on simple data
X, y = make_classification(
    n_samples=200, n_features=2, n_redundant=0,
    n_clusters_per_class=1, class_sep=1.5, random_state=42
)
y = np.where(y == 0, -1, 1)
```

```python

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

# Standardize (crucial for ADALINE)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train ADALINE
adaline = Adaline(learning_rate=0.01, n_epochs=100, random_state
    =42)
adaline.fit(X_train_scaled, y_train, verbose=True)

# Evaluate
train_acc = adaline.score(X_train_scaled, y_train)
test_acc = adaline.score(X_test_scaled, y_test)

print(f"\nTraining accuracy: {train_acc:.2%}")
print(f"Test accuracy: {test_acc:.2%}")
print(f"Final weights: {adaline.weights}")
print(f"Final bias: {adaline.bias:.4f}")
print(f"Final cost: {adaline.cost_history[-1]:.6f}")

# Plot cost history
plt.figure(figsize=(10, 5))
plt.plot(adaline.cost_history, linewidth=2)
plt.xlabel('Epoch')
plt.ylabel('Cost (MSE)')
plt.title('ADALINE Cost Function Evolution')
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 4: Basic ADALINE implementation

5.3.2 Learning Logic Gates with ADALINE

Let's train ADALINE on logic gates and compare with Perceptron:

```python
# Data for logic gates (in -1/1 format)
X_logic = np.array([[0,0], [0,1], [1,0], [1,1]])

# Truth tables in -1/1 format
y_and = np.array([-1, -1, -1, 1])    # AND
y_or = np.array([-1, 1, 1, 1])       # OR
y_nand = np.array([1, 1, 1, -1])     # NAND
y_nor = np.array([1, -1, -1, -1])    # NOR
y_xor = np.array([-1, 1, 1, -1])     # XOR

def train_and_evaluate_adaline(X, y, gate_name, learning_rate
    =0.1, epochs=50):
```

```
    """Train ADALINE on logic gate and evaluate"""

    print(f"\n{'='*50}")
    print(f"Training ADALINE on {gate_name} gate")
    print('='*50)

    # For logic gates, we don't need standardization
    adaline = Adaline(learning_rate=learning_rate, n_epochs=
        epochs, random_state=42)
    adaline.fit(X, y, verbose=False)

    # Get linear outputs (before sign)
    linear_outputs = adaline.activation(X)
    predictions = adaline.predict(X)

    print(f"Weights: {adaline.weights}")
    print(f"Bias: {adaline.bias:.4f}")
    print(f"Final cost: {adaline.cost_history[-1]:.6f}")

    print("\nTruth Table:")
    print("A B | Target | Linear Out | Prediction | Status")
    print("----+--------+------------+------------+-------")
    for i, (x, target, linear, pred) in enumerate(zip(X, y,
        linear_outputs, predictions)):
        status = "OK" if target == pred else "FAIL"
        target_val = 1 if target == 1 else 0
        pred_val = 1 if pred == 1 else 0
        print(f"{int(x[0])} {int(x[1])} |   {target_val}    |   {
            linear:6.4f}   |     {pred_val}     |  {status}")

    accuracy = np.mean(predictions == y)
    print(f"\nAccuracy: {accuracy:.2%}")

    # Plot cost evolution
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(adaline.cost_history, linewidth=2)
    plt.xlabel('Epoch')
    plt.ylabel('Cost')
    plt.title(f'{gate_name} Gate - Cost Evolution')
    plt.grid(True, alpha=0.3)

    # Plot weight evolution
    plt.subplot(1, 2, 2)
    weights_history = np.array(adaline.weights_history)
    plt.plot(weights_history[:, 0], label='Weight 1', linewidth
        =2)
    plt.plot(weights_history[:, 1], label='Weight 2', linewidth
        =2)
    plt.xlabel('Update step')
```

```
    plt.ylabel('Weight value')
    plt.title(f'{gate_name} Gate - Weight Evolution')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    return adaline

# Train on different gates
print("\n" + "="*60)
print("ADALINE ON LOGIC GATES")
print("="*60)

a_and = train_and_evaluate_adaline(X_logic, y_and, "AND",
   learning_rate=0.1, epochs=50)
a_or = train_and_evaluate_adaline(X_logic, y_or, "OR",
   learning_rate=0.1, epochs=50)
a_nand = train_and_evaluate_adaline(X_logic, y_nand, "NAND",
   learning_rate=0.1, epochs=50)
a_nor = train_and_evaluate_adaline(X_logic, y_nor, "NOR",
   learning_rate=0.1, epochs=50)
a_xor = train_and_evaluate_adaline(X_logic, y_xor, "XOR",
   learning_rate=0.1, epochs=100)
```

5.3.3 Comparing ADALINE and Perceptron on XOR

Let's compare how ADALINE and Perceptron behave on the non-separable XOR problem:

```
# Train both models on XOR
p_xor = Perceptron(learning_rate=0.1, n_epochs=100, random_state
   =42)
p_xor.fit(X_logic, y_xor, verbose=False)

a_xor = Adaline(learning_rate=0.1, n_epochs=100, random_state=42)
a_xor.fit(X_logic, y_xor, verbose=False)

# Compare
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.scatter(X_logic[:, 0], X_logic[:, 1], c=y_xor, cmap='bwr', s
   =200, edgecolors='black')
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('XOR Problem')
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 2)
```

```
plt.plot(p_xor.errors_history, linewidth=2, label='Perceptron errors', color='blue')
plt.xlabel('Epoch')
plt.ylabel('Number of errors')
plt.title('Perceptron on XOR')
plt.grid(True, alpha=0.3)
plt.ylim(-0.5, 4.5)

plt.subplot(1, 3, 3)
plt.plot(a_xor.cost_history, linewidth=2, label='ADALINE cost', color='red')
plt.xlabel('Epoch')
plt.ylabel('Cost (MSE)')
plt.title('ADALINE on XOR')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n" + "="*60)
print("PERCEPTRON VS ADALINE ON XOR")
print("="*60)
print(f"Perceptron - Final errors: {p_xor.errors_history[-1]}/4")
print(f"ADALINE - Final cost: {a_xor.cost_history[-1]:.6f}")
print(f"ADALINE - Final linear outputs:")
linear_out = a_xor.activation(X_logic)
for (x1,x2), out in zip(X_logic, linear_out):
    print(f"  ({x1},{x2}) -> {out:.4f} -> {1 if out>=0 else 0}")

print("\nCONCLUSION: ADALINE finds a linear approximation even when")
print("perfect classification is impossible. The cost decreases smoothly.")
```

5.3.4 Effect of Learning Rate on ADALINE

Let's explore how learning rate affects convergence:

```
def compare_adaline_learning_rates(X, y, learning_rates=[0.0001, 0.001, 0.01, 0.1, 1.0], epochs=100):
    """
    Compare ADALINE performance with different learning rates
    """
    plt.figure(figsize=(15, 12))

    results = {}

    for i, lr in enumerate(learning_rates):
        # Train ADALINE
        a = Adaline(learning_rate=lr, n_epochs=epochs, random_state=42)
```

```
        a.fit(X, y, verbose=False)

        # Plot cost evolution
        plt.subplot(3, 2, i+1)
        plt.plot(a.cost_history, linewidth=2)
        plt.xlabel('Epoch')
        plt.ylabel('Cost')
        plt.title(f'Learning Rate = {lr}')
        plt.grid(True, alpha=0.3)
        plt.ylim(0, min(5, max(a.cost_history)*1.1))

        # Store results
        results[lr] = {
            'final_cost': a.cost_history[-1],
            'min_cost': min(a.cost_history),
            'final_weights': a.weights.copy(),
            'bias': a.bias
        }

    plt.tight_layout()
    plt.show()

    # Print comparison
    print("\n" + "="*70)
    print("LEARNING RATE COMPARISON")
    print("="*70)
    print(f"{'LR':<10} {'Final Cost':<12} {'Min Cost':<12} {'Weights':<25} {'Bias':<10}")
    print("-"*70)

    for lr, res in results.items():
        weights_str = f"[{res['final_weights'][0]:.3f}, {res['final_weights'][1]:.3f}]"
        print(f"{lr:<10.4f} {res['final_cost']:<12.6f} {res['min_cost']:<12.6f} "
              f"{weights_str:<25} {res['bias']:<10.3f}")

    return results

# Test on AND gate
print("\n" + "="*70)
print("TESTING DIFFERENT LEARNING RATES ON AND GATE")
print("="*70)
results_and_lr = compare_adaline_learning_rates(X_logic, y_and,
                                                learning_rates=[0.001, 0.01, 0.1, 0.5, 1.0],
                                                epochs=50)

# Test on more complex data
print("\n" + "="*70)
```

```
print("TESTING DIFFERENT LEARNING RATES ON COMPLEX DATA")
print("="*70)
results_comp_lr = compare_adaline_learning_rates(X_train_scaled, y_train,
                                                 learning_rates=[0.0001, 0.001, 0.01, 0.1, 1.0],
                                                 epochs=100)
```

5.3.5 Importance of Standardization

Let's demonstrate why standardization is crucial for ADALINE:

```
# Create data with different scales
np.random.seed(42)
n_samples = 200

# Feature 1: large scale (0-1000), Feature 2: small scale (0-0.1)
X_unscaled = np.column_stack([
    np.random.rand(n_samples) * 1000,  # Large scale
    np.random.rand(n_samples) * 0.1     # Small scale
])

# Create a linear decision boundary
y_unscaled = np.where(0.01 * X_unscaled[:, 0] + 100 * X_unscaled[:, 1] > 5, 1, -1)

# Split
X_train_u, X_test_u, y_train_u, y_test_u = train_test_split(
    X_unscaled, y_unscaled, test_size=0.3, random_state=42
)

# Standardize
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_u)
X_test_scaled = scaler.transform(X_test_u)

# Train ADALINE on unscaled data (need tiny learning rate)
print("\n" + "="*60)
print("TRAINING ON UNSCALED DATA")
print("="*60)
a_unscaled = Adaline(learning_rate=0.000001, n_epochs=200, random_state=42)
a_unscaled.fit(X_train_u, y_train_u, verbose=False)
test_acc_unscaled = a_unscaled.score(X_test_u, y_test_u)
print(f"Test accuracy (unscaled): {test_acc_unscaled:.2%}")
print(f"Final cost (unscaled): {a_unscaled.cost_history[-1]:.6f}")

# Train ADALINE on scaled data (can use normal learning rate)
```

```
print("\n" + "="*60)
print("TRAINING ON SCALED DATA")
print("="*60)
a_scaled = Adaline(learning_rate=0.01, n_epochs=200, random_state=42)
a_scaled.fit(X_train_scaled, y_train_u, verbose=False)
test_acc_scaled = a_scaled.score(X_test_scaled, y_test_u)
print(f"Test accuracy (scaled): {test_acc_scaled:.2%}")
print(f"Final cost (scaled): {a_scaled.cost_history[-1]:.6f}")

# Compare convergence
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(a_unscaled.cost_history, linewidth=2, label='Unscaled')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.title('Without Standardization (slow convergence)')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(a_scaled.cost_history, linewidth=2, label='Scaled', color='green')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.title('With Standardization (fast convergence)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n" + "="*60)
print("CONCLUSION")
print("="*60)
print("Standardization is crucial for ADALINE because:")
print("1. Features with different scales cause uneven updates")
print("2. The cost function becomes elongated (hard to optimize)")
print("3. You need very small learning rates for unscaled data")
print("4. Standardization leads to faster, more stable convergence")
```

5.3.6 Batch vs Stochastic vs Mini-Batch Gradient Descent

Let's compare different gradient descent variants:

```
class AdalineSGD(Adaline):
    """ADALINE with Stochastic Gradient Descent (online learning)
       """

```

```
    def __init__(self, learning_rate=0.01, n_epochs=50,
        random_state=None):
        super().__init__(learning_rate, n_epochs, batch_size=1,
            random_state=random_state)

class AdalineMiniBatch(Adaline):
    """ADALINE with Mini-Batch Gradient Descent"""

    def __init__(self, learning_rate=0.01, n_epochs=50,
        batch_size=32, random_state=None):
        super().__init__(learning_rate, n_epochs, batch_size=
            batch_size, random_state=random_state)

class AdalineBatch(Adaline):
    """ADALINE with Batch Gradient Descent"""

    def __init__(self, learning_rate=0.01, n_epochs=50,
        random_state=None):
        super().__init__(learning_rate, n_epochs, batch_size='
            batch', random_state=random_state)

# Compare all three variants
X_comp, y_comp = make_classification(
    n_samples=1000, n_features=2, n_redundant=0,
    n_clusters_per_class=1, class_sep=1.2, random_state=42
)
y_comp = np.where(y_comp == 0, -1, 1)

scaler = StandardScaler()
X_comp_scaled = scaler.fit_transform(X_comp)

# Train all variants
print("\n" + "="*70)
print("GRADIENT DESCENT VARIANTS COMPARISON")
print("="*70)

a_batch = AdalineBatch(learning_rate=0.1, n_epochs=100,
    random_state=42)
a_batch.fit(X_comp_scaled, y_comp, verbose=False)
print(f"Batch GD - Final cost: {a_batch.cost_history[-1]:.6f}")

a_sgd = AdalineSGD(learning_rate=0.1, n_epochs=100, random_state
    =42)
a_sgd.fit(X_comp_scaled, y_comp, verbose=False)
print(f"SGD - Final cost: {a_sgd.cost_history[-1]:.6f}")

a_mini = AdalineMiniBatch(learning_rate=0.1, n_epochs=100,
    batch_size=32, random_state=42)
a_mini.fit(X_comp_scaled, y_comp, verbose=False)
print(f"Mini-Batch (32) - Final cost: {a_mini.cost_history[-1]:.6
    f}")
```

```
45
46  # Plot comparison
47  plt.figure(figsize=(15, 5))
48
49  plt.subplot(1, 3, 1)
50  plt.plot(a_batch.cost_history, linewidth=2, label='Batch GD',
        color='blue')
51  plt.xlabel('Epoch')
52  plt.ylabel('Cost')
53  plt.title('Batch Gradient Descent')
54  plt.grid(True, alpha=0.3)
55
56  plt.subplot(1, 3, 2)
57  plt.plot(a_sgd.cost_history, linewidth=2, label='SGD', color='
        green')
58  plt.xlabel('Epoch')
59  plt.ylabel('Cost')
60  plt.title('Stochastic Gradient Descent')
61  plt.grid(True, alpha=0.3)
62
63  plt.subplot(1, 3, 3)
64  plt.plot(a_mini.cost_history, linewidth=2, label='Mini-Batch',
        color='red')
65  plt.xlabel('Epoch')
66  plt.ylabel('Cost')
67  plt.title('Mini-Batch Gradient Descent')
68  plt.grid(True, alpha=0.3)
69
70  plt.tight_layout()
71  plt.show()
72
73  print("\n" + "="*70)
74  print("COMPARISON TABLE")
75  print("="*70)
76  print(f"{'Method':<15} {'Pros':<40} {'Cons':<30}")
77  print("-"*85)
78  print(f"{'Batch GD':<15} {'Stable convergence, accurate gradient
        ':<40} {'Slow for large datasets':<30}")
79  print(f"{'SGD':<15} {'Fast, can escape local minima':<40} {'Noisy
         updates, oscillates':<30}")
80  print(f"{'Mini-Batch':<15} {'Best of both worlds':<40} {'Need to
        tune batch size':<30}")
```

5.3.7 ADALINE for Regression

ADALINE can be easily adapted for regression tasks:

```
1  class AdalineRegressor(Adaline):
2      """ADALINE adapted for regression (no sign function)"""
3
4      def predict(self, X):
```

```
        """Regression output (continuous)"""
        return self.net_input(X)

    def score(self, X, y):
        """R^2 score for regression"""
        y_pred = self.predict(X)
        ss_res = np.sum((y - y_pred) ** 2)
        ss_tot = np.sum((y - np.mean(y)) ** 2)
        return 1 - (ss_res / ss_tot) if ss_tot != 0 else 0

# Generate regression data
X_reg, y_reg = make_regression(
    n_samples=300, n_features=1, noise=10, random_state=42
)
y_reg = y_reg.astype(float)

# Split
X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg,
    test_size=0.2, random_state=42)

# Standardize
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)
y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1, 1)).
   ravel()

# Train ADALINE regressor
a_reg = AdalineRegressor(learning_rate=0.01, n_epochs=200,
   random_state=42)
a_reg.fit(X_train_scaled, y_train_scaled, verbose=False)

# Predict and inverse transform
y_train_pred_scaled = a_reg.predict(X_train_scaled)
y_test_pred_scaled = a_reg.predict(X_test_scaled)

y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled.
   reshape(-1, 1)).ravel()
y_test_pred = scaler_y.inverse_transform(y_test_pred_scaled.
   reshape(-1, 1)).ravel()

# Calculate R^2 scores
train_r2 = a_reg.score(X_train_scaled, y_train_scaled)
test_r2 = a_reg.score(X_test_scaled, y_test_scaled)
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)

print("\n" + "="*60)
print("ADALINE FOR REGRESSION")
```

```
print("="*60)
print(f"Training R^2: {train_r2:.4f}")
print(f"Test R^2: {test_r2:.4f}")
print(f"Training MSE: {train_mse:.4f}")
print(f"Test MSE: {test_mse:.4f}")
print(f"Weights: {a_reg.weights}")
print(f"Bias: {a_reg.bias:.4f}")

# Plot results
plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.plot(a_reg.cost_history, linewidth=2, color='orange')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.title('Training Cost Evolution')
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 2)
plt.scatter(X_train, y_train, alpha=0.5, label='Training data')
plt.scatter(X_train, y_train_pred, alpha=0.5, label='Predictions')
plt.xlabel('X')
plt.ylabel('y')
plt.title(f'Training (R^2 = {train_r2:.3f})')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 3)
plt.scatter(X_test, y_test, alpha=0.5, label='Test data')
plt.scatter(X_test, y_test_pred, alpha=0.5, label='Predictions')
# Plot regression line
X_line = np.linspace(X_test.min(), X_test.max(), 100).reshape(-1, 1)
X_line_scaled = scaler_X.transform(X_line)
y_line_scaled = a_reg.predict(X_line_scaled)
y_line = scaler_y.inverse_transform(y_line_scaled.reshape(-1, 1)).ravel()
plt.plot(X_line, y_line, 'r-', linewidth=2, label='ADALINE line')
plt.xlabel('X')
plt.ylabel('y')
plt.title(f'Test (R^2 = {test_r2:.3f})')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

5.3.8 ADALINE on Iris Dataset

Let's apply ADALINE to the Iris dataset:

```python
from sklearn.datasets import load_iris
from sklearn.metrics import confusion_matrix,
    classification_report
import seaborn as sns

# Load Iris dataset (2 classes only)
iris = load_iris()
X_iris = iris.data[:100, :2]  # First 2 classes, first 2 features
y_iris = iris.target[:100]
y_iris = np.where(y_iris == 0, -1, 1)  # Convert to -1/1

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X_iris, y_iris, test_size=0.3, random_state=42
)

# Standardize
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train ADALINE with different learning rates
learning_rates = [0.001, 0.01, 0.1, 0.5]
best_acc = 0
best_lr = None
best_a = None

plt.figure(figsize=(15, 10))

for i, lr in enumerate(learning_rates):
    a = Adaline(learning_rate=lr, n_epochs=200, random_state=42)
    a.fit(X_train_scaled, y_train, verbose=False)

    train_acc = a.score(X_train_scaled, y_train)
    test_acc = a.score(X_test_scaled, y_test)

    plt.subplot(2, 2, i+1)
    plt.plot(a.cost_history, linewidth=2)
    plt.xlabel('Epoch')
    plt.ylabel('Cost')
    plt.title(f'LR={lr} - Train: {train_acc:.2%}, Test: {test_acc
        :.2%}')
    plt.grid(True, alpha=0.3)

    if test_acc > best_acc:
        best_acc = test_acc
        best_lr = lr
        best_a = a

plt.tight_layout()
plt.show()
```

```
50
51  print("\n" + "="*60)
52  print("ADALINE ON IRIS DATASET")
53  print("="*60)
54  print(f"Best learning rate: {best_lr}")
55  print(f"Best test accuracy: {best_acc:.2%}")
56  print(f"Final weights: {best_a.weights}")
57  print(f"Final bias: {best_a.bias:.4f}")
58  print(f"Final cost: {best_a.cost_history[-1]:.6f}")
59
60  # Confusion matrix
61  test_pred = best_a.predict(X_test_scaled)
62  cm = confusion_matrix(y_test, test_pred)
63
64  plt.figure(figsize=(8, 6))
65  sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
66              xticklabels=['Setosa', 'Versicolor'],
67              yticklabels=['Setosa', 'Versicolor'])
68  plt.title(f'Confusion Matrix - ADALINE on Iris (LR={best_lr})')
69  plt.xlabel('Predicted')
70  plt.ylabel('Actual')
71  plt.show()
72
73  # Decision boundary visualization function
74  def plot_adaline_decision_boundary(adaline, X, y, scaler=None,
        title="ADALINE Decision Boundary"):
75      """Plot decision boundary for ADALINE"""
76
77      if scaler:
78          X_plot = X
79      else:
80          X_plot = X
81
82      # Create mesh grid
83      x_min, x_max = X_plot[:, 0].min() - 0.5, X_plot[:, 0].max() +
            0.5
84      y_min, y_max = X_plot[:, 1].min() - 0.5, X_plot[:, 1].max() +
            0.5
85      xx, yy = np.meshgrid(
86          np.linspace(x_min, x_max, 200),
87          np.linspace(y_min, y_max, 200)
88      )
89
90      # Predict on grid
91      Z = adaline.predict(np.c_[xx.ravel(), yy.ravel()])
92      Z = Z.reshape(xx.shape)
93
94      # Plot
95      plt.figure(figsize=(10, 8))
96      plt.contourf(xx, yy, Z, alpha=0.3, levels=[-1, 0, 1], colors
            =['blue', 'red'])
```

```
    plt.contour(xx, yy, Z, levels=[0], colors='black', linewidths=2)

    # Plot data points
    scatter = plt.scatter(X_plot[:, 0], X_plot[:, 1], c=y,
                          cmap='bwr', edgecolors='black', s=100, alpha=0.8)

    # Add weights and bias info
    w = adaline.weights
    b = adaline.bias
    plt.text(x_min + 0.5, y_max - 0.5,
             f'w = [{w[0]:.2f}, {w[1]:.2f}]\nb = {b:.2f}\ncost = {adaline.cost_history[-1]:.4f}',
             bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.7))

    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.title(title)
    plt.colorbar(scatter)
    plt.grid(True, alpha=0.3)
    plt.show()

plot_adaline_decision_boundary(best_a, X_test_scaled, y_test, scaler,
                               f"ADALINE on Iris (LR={best_lr}, Acc={best_acc:.2%})")
```

5.3.9 Learning with Momentum

Adding momentum can accelerate convergence:

```
class AdalineMomentum(Adaline):
    """ADALINE with momentum for faster convergence"""

    def __init__(self, learning_rate=0.01, n_epochs=50, momentum=0.9, random_state=None):
        super().__init__(learning_rate, n_epochs, random_state=random_state)
        self.momentum = momentum
        self.weight_update = None
        self.bias_update = None

    def fit(self, X, y, verbose=True):
        n_samples, n_features = X.shape

        self._initialize_weights(n_features)
        self.cost_history = []
        self.weights_history = [self.weights.copy()]

```

```
        # Initialize momentum terms
        self.weight_update = np.zeros(n_features)
        self.bias_update = 0.0

        for epoch in range(self.n_epochs):
            # Shuffle data
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            epoch_cost = 0

            for xi, target in zip(X_shuffled, y_shuffled):
                # Compute output
                output = self.activation(xi.reshape(1, -1))[0]

                # Compute error
                error = target - output

                # Compute gradient
                weight_grad = error * xi
                bias_grad = error

                # Update with momentum
                self.weight_update = self.momentum * self.
                    weight_update + self.lr * weight_grad
                self.bias_update = self.momentum * self.
                    bias_update + self.lr * bias_grad

                self.weights += self.weight_update
                self.bias += self.bias_update

                epoch_cost += error**2

            epoch_cost /= (2 * n_samples)
            self.cost_history.append(epoch_cost)
            self.weights_history.append(self.weights.copy())

            if verbose and (epoch + 1) % 10 == 0:
                print(f"Epoch {epoch+1}/{self.n_epochs}, Cost: {
                    epoch_cost:.6f}")

        return self

# Compare with and without momentum
print("\n" + "="*60)
print("ADALINE WITH vs WITHOUT MOMENTUM")
print("="*60)

a_standard = Adaline(learning_rate=0.1, n_epochs=100,
    random_state=42)
```

```
a_momentum = AdalineMomentum(learning_rate=0.1, n_epochs=100, momentum=0.9, random_state=42)

a_standard.fit(X_train_scaled, y_train, verbose=False)
a_momentum.fit(X_train_scaled, y_train, verbose=False)

# Plot comparison
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(a_standard.cost_history, linewidth=2, label='Standard', color='blue')
plt.plot(a_momentum.cost_history, linewidth=2, label='With Momentum', color='red')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.title('Convergence Speed Comparison')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
# This is a simplified visualization
plt.bar(['Standard', 'Momentum'],
        [a_standard.cost_history[-1], a_momentum.cost_history[-1]],
        color=['blue', 'red'])
plt.ylabel('Final Cost')
plt.title('Final Cost Comparison')
plt.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print(f"Standard ADALINE - Final cost: {a_standard.cost_history[-1]:.6f}")
print(f"Momentum ADALINE - Final cost: {a_momentum.cost_history[-1]:.6f}")
print(f"Improvement: {(a_standard.cost_history[-1] - a_momentum.cost_history[-1]) / a_standard.cost_history[-1] * 100:.2f}%")
```

5.3.10 Learning Rate Decay

Implementing learning rate decay for better convergence:

```
class AdalineDecay(Adaline):
    """ADALINE with learning rate decay"""

    def __init__(self, learning_rate=0.1, n_epochs=50, decay_rate=0.99, random_state=None):
        super().__init__(learning_rate, n_epochs, random_state=random_state)
```

```
        self.decay_rate = decay_rate
        self.learning_rates = []

    def fit(self, X, y, verbose=True):
        n_samples, n_features = X.shape

        self._initialize_weights(n_features)
        self.cost_history = []
        self.weights_history = [self.weights.copy()]
        self.learning_rates = []

        current_lr = self.lr

        for epoch in range(self.n_epochs):
            # Shuffle data
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            epoch_cost = 0

            for xi, target in zip(X_shuffled, y_shuffled):
                # Compute output
                output = self.activation(xi.reshape(1, -1))[0]

                # Compute error
                error = target - output

                # Update with current learning rate
                self.weights += current_lr * error * xi
                self.bias += current_lr * error

                epoch_cost += error**2

            epoch_cost /= (2 * n_samples)
            self.cost_history.append(epoch_cost)
            self.weights_history.append(self.weights.copy())
            self.learning_rates.append(current_lr)

            # Decay learning rate
            current_lr *= self.decay_rate

            if verbose and (epoch + 1) % 10 == 0:
                print(f"Epoch {epoch+1}/{self.n_epochs}, Cost: {
                    epoch_cost:.6f}, LR: {current_lr:.6f}")

        return self

# Compare constant vs decay
print("\n" + "="*60)
print("ADALINE WITH LEARNING RATE DECAY")
```

```
print("="*60)

a_constant = Adaline(learning_rate=0.1, n_epochs=100, random_state=42)
a_decay = AdalineDecay(learning_rate=0.1, n_epochs=100, decay_rate=0.98, random_state=42)

a_constant.fit(X_train_scaled, y_train, verbose=False)
a_decay.fit(X_train_scaled, y_train, verbose=False)

# Plot comparison
plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.plot(a_constant.cost_history, linewidth=2, label='Constant LR', color='blue')
plt.plot(a_decay.cost_history, linewidth=2, label='Decay LR', color='red')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.title('Cost Evolution')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 2)
plt.plot(a_decay.learning_rates, linewidth=2, color='green')
plt.xlabel('Epoch')
plt.ylabel('Learning Rate')
plt.title('Learning Rate Decay')
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 3)
# Final test accuracy
test_acc_const = a_constant.score(X_test_scaled, y_test)
test_acc_decay = a_decay.score(X_test_scaled, y_test)
plt.bar(['Constant LR', 'Decay LR'], [test_acc_const, test_acc_decay],
        color=['blue', 'red'])
plt.ylabel('Test Accuracy')
plt.title('Final Test Accuracy')
plt.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print(f"Constant LR - Final cost: {a_constant.cost_history[-1]:.6f}, Test acc: {test_acc_const:.2%}")
print(f"Decay LR - Final cost: {a_decay.cost_history[-1]:.6f}, Test acc: {test_acc_decay:.2%}")
```

5.4 Summary of ADALINE

Property	Description
Learning Type	Supervised
Activation (training)	Linear (identity)
Activation (prediction)	Sign (step)
Cost Function	Mean Squared Error (MSE)
Optimization	Gradient Descent
Update Rule	$\Delta w = \eta(y_{\text{target}} - y_{\text{linear}})x$
Convergence	Guaranteed to global minimum (convex)
Advantages	Smooth learning, continuous error, optimal for linear problems
Disadvantages	Sensitive to feature scales, requires standardization
Applications	Classification, regression, adaptive filtering, echo cancellation

Table 10: Summary of ADALINE Properties

Key takeaways:

- ADALINE uses **linear activation during training** and sign for prediction
- It minimizes the **Mean Squared Error** cost function
- The cost function is **convex**, guaranteeing convergence to global minimum
- Updates are **proportional to the error** (not just sign of error)
- **Standardization** is crucial for good performance
- Multiple variants exist: **batch, stochastic, mini-batch** gradient descent
- Can be extended with **momentum** and **learning rate decay**
- Works for both **classification and regression** tasks
- Cannot solve non-linear problems (same limitation as Perceptron)
- Laid the foundation for modern adaptive systems and signal processing

5.5 Additional Exercises

Exercise 4.21 **Adaptive Noise Cancellation:** Research how ADALINE is used for noise cancellation. Implement a simple version that removes noise from a sine wave.

Exercise 4.22 **Comparison with Linear Regression:** Compare ADALINE with scikit-learn's LinearRegression on a regression task. How do the weights compare?

Exercise 4.23 **Regularized ADALINE:** Add L2 regularization (ridge) to ADALINE. Modify the cost function and update rule accordingly.

Exercise 4.24 **Polynomial Features:** Create polynomial features to make XOR linearly separable. Train ADALINE on these features and compare with Perceptron.

Exercise 4.25 **Online Learning Simulation:** Simulate an online learning scenario where data arrives sequentially. Plot how the weights evolve over time.

Exercise 4.26 **ADALINE for Time Series Prediction:** Use ADALINE to predict the next value in a time series. Design the feature vectors appropriately.

Exercise 4.27 **Visualizing the Error Surface:** For 2D data, plot the error surface and show the path of gradient descent during training.

Exercise 4.28 **Comparison of Optimizers:** Implement different gradient descent optimizers (Adam, RMSprop) for ADALINE and compare convergence.

Exercise 4.29 **Early Stopping:** Implement early stopping based on validation set performance to prevent overfitting.

Exercise 4.30 **Weight Decay:** Add weight decay to ADALINE and observe its effect on the learned weights.

General Conclusion

Throughout this lab, we have explored the historical foundations of neural networks:

Model	Year	Learning	Innovation
McCulloch-Pitts	1943	None	First mathematical model of a neuron
Hebb	1949	Unsupervised	First local learning rule
Perceptron	1957	Supervised	First trainable classifier
ADALINE	1960	Supervised	Gradient descent, MSE minimization

Table 11: Summary of historical neural network models

Key Takeaways

- **Linearly separable problems** can be solved by single neurons (McCulloch-Pitts, Perceptron, ADALINE)
- **Non-linearly separable problems** (like XOR) require multi-layer networks
- **Hebbian learning** discovers correlations in data without supervision
- **Perceptron** introduced supervised learning with guaranteed convergence
- **ADALINE** improved learning stability with gradient descent and MSE
- **Standardization** is crucial for gradient-based methods
- These models, though simple, laid the groundwork for modern deep learning

Next Steps

To go further, you can explore:

- Multi-layer Perceptrons and backpropagation
- Non-linear activation functions (tanh, ReLU)
- Deep learning architectures (CNNs, RNNs)
- Modern optimizers (Adam, RMSprop)
- Regularization techniques (dropout, batch normalization)

Important: Remember: Before deep learning, there were decades of fundamental research that made possible what we use today. Understanding these foundations is essential for any serious work in neural networks and artificial intelligence.